

Control Unit UVM Verification

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 RTL design	11
6.1.1 Detailed Description	11
6.2 Modules	11
6.3 Testbench components	11
6.3.1 Detailed Description	11
6.4 Interfaces	12
6.4.1 Detailed Description	12
6.4.2 Function Documentation	12
6.4.2.1 s_interface()	12
6.5 UVM verification components	13
6.5.1 Detailed Description	13
7 Directory Documentation	15
7.1 filter Directory Reference	15
7.1.1 Detailed Description	15
7.1.2 SystemVerilog Doxygen filter	15
7.2 reference_model Directory Reference	15
7.3 rtl Directory Reference	15
7.4 tb Directory Reference	16
7.5 uvm Directory Reference	16
8 Class Documentation	17
8.1 agent Class Reference	17
8.1.1 Detailed Description	18
8.1.2 Member Function Documentation	18
8.1.2.1 build_phase()	18
8.1.2.2 connect_phase()	18
8.1.2.3 new()	18
8.1.2.4 uvm_component_utils()	19

8.1.3 Member Data Documentation	19
8.1.3.1 m_monitor	19
8.1.3.2 m_sequencer	19
8.2 control_unit_sequence Class Reference	19
8.2.1 Detailed Description	20
8.2.2 Member Function Documentation	20
8.2.2.1 body()	20
8.2.2.2 new()	20
8.2.2.3 uvm_object_utils()	21
8.2.3 Member Data Documentation	21
8.2.3.1 num_packets	21
8.3 coverage Class Reference	21
8.3.1 Detailed Description	22
8.3.2 Member Function Documentation	22
8.3.2.1 cg_control_unit()	22
8.3.2.2 new()	22
8.3.2.3 uvm_component_utils()	23
8.3.2.4 write()	23
8.4 driver Class Reference	23
8.4.1 Detailed Description	24
8.4.2 Member Function Documentation	25
8.4.2.1 build_phase()	25
8.4.2.2 drive_item()	25
8.4.2.3 new()	25
8.4.2.4 run_phase()	25
8.4.2.5 uvm_component_utils()	25
8.4.3 Member Data Documentation	25
8.4.3.1 req_pkt	25
8.5 env Class Reference	26
8.5.1 Detailed Description	27
8.5.2 Member Function Documentation	27
8.5.2.1 build_phase()	27
8.5.2.2 connect_phase()	27
8.5.2.3 new()	27
8.5.2.4 uvm_component_utils()	27
8.5.3 Member Data Documentation	27
8.5.3.1 m_coverage	27
8.5.3.2 m_scoreboard	28
8.6 monitor Class Reference	28
8.6.1 Detailed Description	29
8.6.2 Member Function Documentation	29
8.6.2.1 build_phase()	29

8.6.2.2 new()	29
8.6.2.3 run_phase()	29
8.6.2.4 uvm_component_utils()	29
8.6.3 Member Data Documentation	30
8.6.3.1 mon_analysis_port	30
8.7 pkt Class Reference	30
8.7.1 Detailed Description	32
8.7.2 Member Function Documentation	32
8.7.2.1 branch_func3_valid()	32
8.7.2.2 new()	32
8.7.2.3 op_range()	32
8.7.2.4 uvm_field_int() [1/8]	32
8.7.2.5 uvm_field_int() [2/8]	32
8.7.2.6 uvm_field_int() [3/8]	33
8.7.2.7 uvm_field_int() [4/8]	33
8.7.2.8 uvm_field_int() [5/8]	33
8.7.2.9 uvm_field_int() [6/8]	33
8.7.2.10 uvm_field_int() [7/8]	33
8.7.2.11 uvm_field_int() [8/8]	33
8.7.2.12 uvm_object_utils_begin()	33
8.7.3 Member Data Documentation	34
8.7.3.1 i_branch_EX	34
8.7.3.2 i_funcnt_3	34
8.7.3.3 i_funcnt_7_5	34
8.7.3.4 i_jump_EX	34
8.7.3.5 i_op	34
8.7.3.6 i_zero	34
8.7.3.7 o_addr_src_ID	34
8.7.3.8 o_alu_ctrl_ID	35
8.7.3.9 o_alu_src_ID	35
8.7.3.10 o_branch_ID	35
8.7.3.11 o_fence_ID	35
8.7.3.12 o_imm_src_ID	35
8.7.3.13 o_jump_ID	35
8.7.3.14 o_mem_write_ID	35
8.7.3.15 o_pc_src_EX	35
8.7.3.16 o_reg_write_ID	36
8.7.3.17 o_result_src_ID	36
8.8 s_interface Interface Reference	36
8.8.1 Detailed Description	36
8.9 scoreboard Class Reference	36
8.9.1 Detailed Description	37

8.9.2 Member Function Documentation	37
8.9.2.1 build_phase()	37
8.9.2.2 check_phase()	38
8.9.2.3 new()	38
8.9.2.4 run_phase()	38
8.9.2.5 uvm_component_utils()	38
8.9.2.6 write()	38
8.9.3 Member Data Documentation	38
8.9.3.1 ap_imp	38
8.9.3.2 num_errors	38
8.10 sequencer Class Reference	39
8.10.1 Detailed Description	39
8.10.2 Member Function Documentation	40
8.10.2.1 uvm_component_utils()	40
8.10.3 Member Data Documentation	40
8.10.3.1 parent	40
8.11 test Class Reference	40
8.11.1 Detailed Description	41
8.11.2 Member Function Documentation	41
8.11.2.1 build_phase()	41
8.11.2.2 end_of_elaboration_phase()	42
8.11.2.3 new()	42
8.11.2.4 run_phase()	42
8.11.2.5 uvm_component_utils()	42
8.11.3 Member Data Documentation	42
8.11.3.1 m_sequence	42
9 File Documentation	43
9.1 filter/README.md File Reference	43
9.2 reference_model/reference_model.c File Reference	43
9.2.1 Detailed Description	44
9.2.2 Macro Definition Documentation	44
9.2.2.1 ALU_ADD	44
9.2.2.2 ALU_AND	45
9.2.2.3 ALU_BEQ	45
9.2.2.4 ALU_BGE	45
9.2.2.5 ALU_BGEU	45
9.2.2.6 ALU_BLT	45
9.2.2.7 ALU_BLTU	45
9.2.2.8 ALU_BNE	45
9.2.2.9 ALU_LUI	45
9.2.2.10 ALU_OR	45

9.2.2.11 ALU_SLL	45
9.2.2.12 ALU_SLT	46
9.2.2.13 ALU_SLTU	46
9.2.2.14 ALU_SRA	46
9.2.2.15 ALU_SRL	46
9.2.2.16 ALU_SUB	46
9.2.2.17 ALU_XOR	46
9.2.2.18 IMM_B	46
9.2.2.19 IMM_I	46
9.2.2.20 IMM_J	46
9.2.2.21 IMM_S	46
9.2.2.22 IMM_U	47
9.2.2.23 OP_AUIPC	47
9.2.2.24 OP_BRANCH	47
9.2.2.25 OP_IMM	47
9.2.2.26 OP_JAL	47
9.2.2.27 OP_JALR	47
9.2.2.28 OP_LOAD	47
9.2.2.29 OP_LUI	47
9.2.2.30 OP_REG	47
9.2.2.31 OP_STORE	47
9.2.2.32 RESULT_ALU	48
9.2.2.33 RESULT_MEM	48
9.2.2.34 RESULT_PC4	48
9.2.2.35 RESULT_PCIMM	48
9.2.3 Function Documentation	48
9.2.3.1 control_unit_golden()	48
9.3 rtl/alu_decoder.v File Reference	49
9.3.1 Variable Documentation	50
9.3.1.1 ADD	50
9.3.1.2 AND	50
9.3.1.3 AUIPC	50
9.3.1.4 BEQ	50
9.3.1.5 BGE	50
9.3.1.6 BGEU	50
9.3.1.7 BLT	50
9.3.1.8 BLTU	51
9.3.1.9 BNE	51
9.3.1.10 EBREAK	51
9.3.1.11 ECALL	51
9.3.1.12 FENCE	51
9.3.1.13 i_alu_op	51

9.3.1.14 i_funct_3	51
9.3.1.15 i_funct_7_5	51
9.3.1.16 LUI	51
9.3.1.17 o_alu_ctrl_ID	52
9.3.1.18 OP_ADD	52
9.3.1.19 OP_ADD_SUB	52
9.3.1.20 OP_ARITH	52
9.3.1.21 OP_BRANCH	52
9.3.1.22 OP_LUI	52
9.3.1.23 OR	52
9.3.1.24 SLL	53
9.3.1.25 SLT	53
9.3.1.26 SLTU	53
9.3.1.27 SRA	53
9.3.1.28 SRL	53
9.3.1.29 SUB	53
9.3.1.30 XOR	53
9.4 rtl/control_unit.v File Reference	53
9.4.1 Function Documentation	54
9.4.1.1 U_ALU_DECODER()	54
9.4.1.2 U_OP_DECODER()	54
9.4.2 Variable Documentation	54
9.4.2.1 alu_op	54
9.4.2.2 i_branch_EX	55
9.4.2.3 i_funct_3	55
9.4.2.4 i_funct_7_5	55
9.4.2.5 i_jump_EX	55
9.4.2.6 i_op	55
9.4.2.7 i_zero	55
9.4.2.8 o_addr_src_ID	55
9.4.2.9 o_alu_ctrl_ID	55
9.4.2.10 o_alu_src_ID	55
9.4.2.11 o_branch_ID	56
9.4.2.12 o_fence_ID	56
9.4.2.13 o_imm_src_ID	56
9.4.2.14 o_jump_ID	56
9.4.2.15 o_mem_write_ID	56
9.4.2.16 o_pc_src_EX	56
9.4.2.17 o_reg_write_ID	56
9.4.2.18 o_result_src_ID	56
9.5 rtl/control_unit_wrapper.sv File Reference	57
9.5.1 Detailed Description	57

9.6 rtl/op_decoder.v File Reference	57
9.6.1 Variable Documentation	58
9.6.1.1 B_type	58
9.6.1.2 B_TYPE_BRANCH	58
9.6.1.3 i_funct_3	58
9.6.1.4 i_funct_7_5	58
9.6.1.5 i_op	58
9.6.1.6 l_type	58
9.6.1.7 l_TYPE_ARITH	58
9.6.1.8 l_TYPE_FENCE	58
9.6.1.9 l_TYPE_JALR	58
9.6.1.10 l_TYPE_LOAD	58
9.6.1.11 l_TYPE_SYSTEM	59
9.6.1.12 J_type	59
9.6.1.13 J_TYPE_JAL	59
9.6.1.14 o_addr_src_ID	59
9.6.1.15 o_alu_op	59
9.6.1.16 o_alu_src_ID	59
9.6.1.17 o_branch_ID	59
9.6.1.18 o_fence_ID	59
9.6.1.19 o_imm_src_ID	59
9.6.1.20 o_jump_ID	60
9.6.1.21 o_mem_write_ID	60
9.6.1.22 o_reg_write_ID	60
9.6.1.23 o_result_src_ID	60
9.6.1.24 OP_ADD	60
9.6.1.25 OP_ADD_SUB	60
9.6.1.26 OP_ARITH	60
9.6.1.27 OP_BRANCH	60
9.6.1.28 OP_LUI	61
9.6.1.29 R_TYPE	61
9.6.1.30 S_TYPE	61
9.6.1.31 S_type	61
9.6.1.32 U_type	61
9.6.1.33 U_TYPE_AUIPC	61
9.6.1.34 U_TYPE_LUI	61
9.7 tb/s_interface.sv File Reference	61
9.7.1 Detailed Description	61
9.8 tb/tb_top.sv File Reference	62
9.8.1 Detailed Description	62
9.8.2 Function Documentation	62
9.8.2.1 run_test()	62

9.8.2.2 U_CONTROL_UNIT_WRAPPER()	62
9.8.2.3 vif()	62
9.8.3 Variable Documentation	62
9.8.3.1 forever	62
9.8.3.2 rst	63
9.9 uvm/agent.sv File Reference	63
9.9.1 Detailed Description	63
9.10 agent.sv	63
9.11 uvm/control_unit_pkg.sv File Reference	64
9.11.1 Detailed Description	64
9.12 uvm/control_unit_sequence.sv File Reference	64
9.12.1 Detailed Description	65
9.13 control_unit_sequence.sv	65
9.14 uvm/coverage.sv File Reference	66
9.14.1 Detailed Description	66
9.15 coverage.sv	66
9.16 uvm/driver.sv File Reference	67
9.16.1 Detailed Description	68
9.17 driver.sv	68
9.18 uvm/env.sv File Reference	68
9.18.1 Detailed Description	69
9.19 env.sv	69
9.20 uvm/monitor.sv File Reference	70
9.20.1 Detailed Description	70
9.21 monitor.sv	70
9.22 uvm/pkt.sv File Reference	71
9.22.1 Detailed Description	72
9.23 pkt.sv	72
9.24 uvm/scoreboard.sv File Reference	72
9.24.1 Detailed Description	73
9.24.2 Function Documentation	73
9.24.2.1 control_unit_golden()	73
9.25 scoreboard.sv	74
9.26 uvm/sequencer.sv File Reference	76
9.26.1 Detailed Description	76
9.27 sequencer.sv	77
9.28 uvm/test.sv File Reference	77
9.28.1 Detailed Description	77
9.29 test.sv	78
Index	79

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

RTL design	11
Modules	11
Testbench components	11
Interfaces	12
UVM verification components	13

Chapter 2

Directory Hierarchy

2.1 Directories

filter	15
reference_model	15
reference_model.c	43
rtl	15
alu_decoder.v	49
control_unit.v	53
control_unit_wrapper.sv	57
op_decoder.v	57
tb	16
s_interface.sv	61
tb_top.sv	62
uvm	16
agent.sv	63
control_unit_pkg.sv	64
control_unit_sequence.sv	64
coverage.sv	66
driver.sv	67
env.sv	68
monitor.sv	70
pkt.sv	71
scoreboard.sv	72
sequencer.sv	76
test.sv	77

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

s_interface	36
uvm_agent	
agent	17
uvm_driver	
driver	23
uvm_env	
env	26
uvm_monitor	
monitor	28
uvm_scoreboard	
scoreboard	36
uvm_sequence	
control_unit_sequence	19
uvm_sequence_item	
pkt	30
uvm_sequencer	
sequencer	39
uvm_subscriber	
coverage	21
uvm_test	
test	40

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

agent	UVM agent that connects the sequencer, driver, and monitor	17
control_unit_sequence	UVM sequence that drives control-unit transactions until coverage reaches the target	19
coverage	UVM coverage subscriber for control-unit transaction sampling	21
driver	UVM driver for control-unit transactions	23
env	Top-level UVM environment for the control-unit verification bench	26
monitor	UVM monitor for control-unit transaction observation	28
pkt	Sequence item that models one control-unit decode (stimulus + expected response fields)	30
s_interface	Control-unit stimulus/response interface shared by the DUT and the UVM environment	36
scoreboard	UVM scoreboard for control-unit functional checking	36
sequencer	UVM sequencer for control-unit transactions	39
test	Top-level UVM test for the control-unit verification bench	40

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

reference_model/ reference_model.c	
Golden reference model for the control unit, called from the UVM scoreboard via DPI-C	43
rtl/ alu_decoder.v	49
rtl/ control_unit.v	53
rtl/ control_unit_wrapper.sv	
Thin combinational wrapper around control_unit , used as the DUT	57
rtl/ op_decoder.v	57
tb/ s_interface.sv	
Control-unit stimulus/response interface shared by the DUT and the UVM environment	61
tb/ tb_top.sv	
Top-level testbench that instantiates the DUT, interface, and UVM test	62
uvm/ agent.sv	
UVM agent that bundles the sequencer, driver, and monitor	63
uvm/ control_unit_pkg.sv	
Package that includes the UVM classes used by the control-unit verification environment	64
uvm/ control_unit_sequence.sv	
UVM sequence that drives control-unit transactions until coverage reaches the target	64
uvm/ coverage.sv	
Coverage collector for control-unit transactions	66
uvm/ driver.sv	
UVM driver that translates control-unit transactions into interface signal activity	67
uvm/ env.sv	
UVM environment that instantiates the agent, scoreboard, and coverage collector	68
uvm/ monitor.sv	
UVM monitor that samples control-unit transactions and publishes them to subscribers	70
uvm/ pkt.sv	
Transaction item used to model one control-unit decode in the UVM environment	71
uvm/ scoreboard.sv	
Scoreboard that checks control-unit transactions against the golden reference model	72
uvm/ sequencer.sv	
UVM sequencer that provides transactions to the driver	76
uvm/ test.sv	
Top-level UVM test that builds the environment and starts the main sequence	77

Chapter 6

Topic Documentation

6.1 RTL design

Register-transfer-level implementation of the control-unit DUT.

Files

- file [control_unit_wrapper.sv](#)
Thin combinational wrapper around [control_unit](#), used as the DUT.

6.1.1 Detailed Description

Register-transfer-level implementation of the control-unit DUT.

6.2 Modules

6.3 Testbench components

Interface and top-level testbench modules used by the verification environment.

Files

- file [s_interface.sv](#)
Control-unit stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

6.3.1 Detailed Description

Interface and top-level testbench modules used by the verification environment.

6.4 Interfaces

Functions

- interface [s_interface](#) (input logic clk, input logic reset)

6.4.1 Detailed Description

6.4.2 Function Documentation

6.4.2.1 s_interface()

```
interface s_interface (
    input logic clk,
    input logic reset)
```

< RV32I base opcode, instr[6:2] ([i_op](#)).

< instr[14:12] ([i_funct_3](#)).

< instr[30] ([i_funct_7_5](#)).

< Registered o_branch_ID at Execute ([i_branch_EX](#)).

< Registered o_jump_ID at Execute ([i_jump_EX](#)).

< ALU branch-comparison result ([i_zero](#)).

< [o_pc_src_EX](#), driven by the DUT.

< [o_jump_ID](#), driven by the DUT.

< [o_branch_ID](#), driven by the DUT.

< [o_reg_write_ID](#), driven by the DUT.

< [o_result_src_ID](#), driven by the DUT.

< [o_mem_write_ID](#), driven by the DUT.

< [o_alu_ctrl_ID](#), driven by the DUT.

< [o_alu_src_ID](#), driven by the DUT.

< [o_addr_src_ID](#), driven by the DUT.

< [o_imm_src_ID](#), driven by the DUT.

< [o_fence_ID](#), driven by the DUT.

< Pulsed high by the driver while a transaction is valid.

Modport used by the UVM driver: drives the stimulus signals.

Modport used by the UVM monitor: observes every signal, drives none.

Modport used by the [control_unit_wrapper](#) DUT: consumes the stimulus, drives the results.

6.5 UVM verification components

UVM classes and utilities used to verify the control unit.

Files

- file [reference_model.c](#)
Golden reference model for the control unit, called from the UVM scoreboard via DPI-C.
- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [control_unit_sequence.sv](#)
UVM sequence that drives control-unit transactions until coverage reaches the target.
- file [coverage.sv](#)
Coverage collector for control-unit transactions.
- file [driver.sv](#)
UVM driver that translates control-unit transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples control-unit transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one control-unit decode in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks control-unit transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

6.5.1 Detailed Description

UVM classes and utilities used to verify the control unit.

Chapter 7

Directory Documentation

7.1 filter Directory Reference

7.1.1 Detailed Description

7.1.2 SystemVerilog Doxygen filter

`idv_doxyfilter_sv.pl` converts SystemVerilog into C++-like code so Doxygen can parse it.

- **Author:** Sean O'Boyle, Intelligent Design Verification
- **Source:** <https://github.com/SeanOBoyle/DoxygenFilterSystemVerilog>
- **License:** GNU GPL v3 or later (see the header of the script). The file is included here unmodified.

It is wired up through `FILTER_PATTERNS` in `../Doxyfile`; run `doxygen` from the parent directory.

7.2 reference_model Directory Reference

Files

- file [reference_model.c](#)
Golden reference model for the control unit, called from the UVM scoreboard via DPI-C.

7.3 rtl Directory Reference

Files

- file [alu_decoder.v](#)
- file [control_unit.v](#)
- file [control_unit_wrapper.sv](#)
Thin combinational wrapper around [control_unit](#), used as the DUT.
- file [op_decoder.v](#)

7.4 tb Directory Reference

Files

- file [s_interface.sv](#)
Control-unit stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

7.5 uvm Directory Reference

Files

- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [control_unit_pkg.sv](#)
Package that includes the UVM classes used by the control-unit verification environment.
- file [control_unit_sequence.sv](#)
UVM sequence that drives control-unit transactions until coverage reaches the target.
- file [coverage.sv](#)
Coverage collector for control-unit transactions.
- file [driver.sv](#)
UVM driver that translates control-unit transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [monitor.sv](#)
UVM monitor that samples control-unit transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one control-unit decode in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks control-unit transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

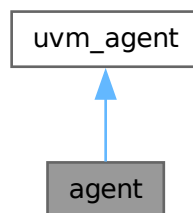
Chapter 8

Class Documentation

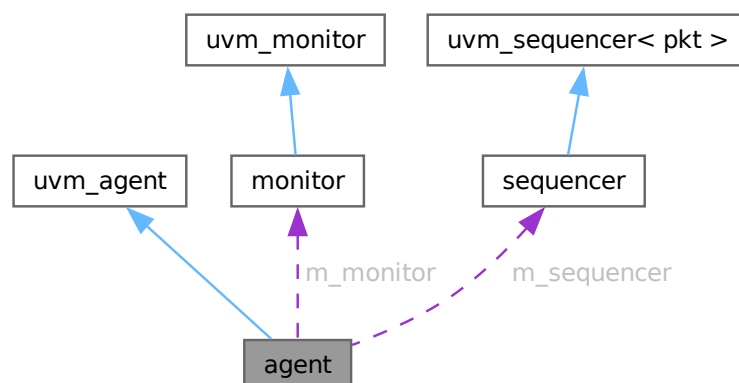
8.1 agent Class Reference

UVM agent that connects the sequencer, driver, and monitor.

Inheritance diagram for agent:



Collaboration diagram for agent:



Public Member Functions

- [uvm_component_utils](#) ([agent](#)) [driver](#) m_driver
Drives sequence items onto the DUT interface (null when passive).
- [new](#) (string name="agent", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Builds the sequencer/driver (active agent only) and the monitor.
- virtual void [connect_phase](#) (uvm_phase phase)
Connects the driver's seq_item port to the sequencer (active agent only).

Public Attributes

- [monitor](#) m_monitor
Samples DUT activity and publishes it (always created).
- [sequencer](#) m_sequencer
Arbitrates sequence items for the driver (null when passive).

8.1.1 Detailed Description

UVM agent that connects the sequencer, driver, and monitor.

Coordinates the active and passive verification components for the control-unit testbench and exposes the common sequence and analysis interfaces. The sequencer and driver are only created when the agent is active; the monitor is always created.

8.1.2 Member Function Documentation

8.1.2.1 build_phase()

```
virtual void agent::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Builds the sequencer/driver (active agent only) and the monitor.

8.1.2.2 connect_phase()

```
virtual void agent::connect_phase (
    uvm_phase phase) [inline], [virtual]
```

Connects the driver's seq_item port to the sequencer (active agent only).

8.1.2.3 new()

```
agent::new (
    string name = "agent",
    uvm_component parent = null) [inline]
```

8.1.2.4 uvm_component_utils()

```
agent::uvm_component_utils (
    agent )
```

Drives sequence items onto the DUT interface (null when passive).

8.1.3 Member Data Documentation

8.1.3.1 m_monitor

```
monitor agent::m_monitor
```

Samples DUT activity and publishes it (always created).

8.1.3.2 m_sequencer

```
sequencer agent::m_sequencer
```

Arbitrates sequence items for the driver (null when passive).

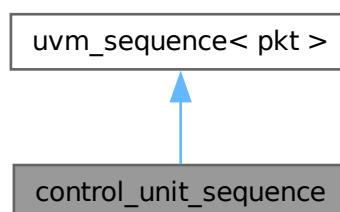
The documentation for this class was generated from the following file:

- [uvm/agent.sv](#)

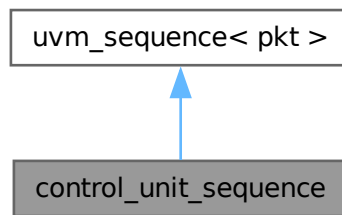
8.2 control_unit_sequence Class Reference

UVM sequence that drives control-unit transactions until coverage reaches the target.

Inheritance diagram for control_unit_sequence:



Collaboration diagram for control_unit_sequence:



Public Member Functions

- `uvm_object_utils (control_unit_sequence)` real current_coverage=0
Latest coverage percentage reported by the coverage collector.
- `new` (string name="control_unit_sequence")
- virtual task `body ()`
Sends randomized control-unit transactions until the coverage collector reports 100%.

Public Attributes

- int `num_packets` = 0
Number of transactions sent so far in this sequence.

8.2.1 Detailed Description

UVM sequence that drives control-unit transactions until coverage reaches the target.

Repeatedly randomizes and sends a transaction, polling the coverage collector after each one, until 100% coverage is reached. Reads the coverage percentage from the configuration database key "cov_status".

8.2.2 Member Function Documentation

8.2.2.1 body()

```
virtual task control_unit_sequence::body () [inline], [virtual]
```

Sends randomized control-unit transactions until the coverage collector reports 100%.

8.2.2.2 new()

```
control_unit_sequence::new (
    string name = "control_unit_sequence") [inline]
```

8.2.2.3 uvm_object_utils()

```
control_unit_sequence::uvm_object_utils (
    control_unit_sequence ) [pure virtual]
```

Latest coverage percentage reported by the coverage collector.

8.2.3 Member Data Documentation

8.2.3.1 num_packets

```
int control_unit_sequence::num_packets = 0
```

Number of transactions sent so far in this sequence.

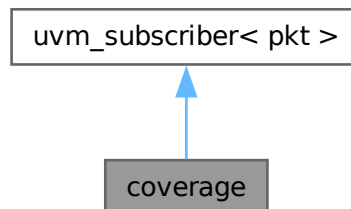
The documentation for this class was generated from the following file:

- [uvm/control_unit_sequence.sv](#)

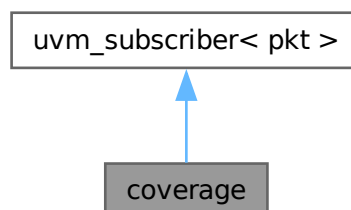
8.3 coverage Class Reference

UVM coverage subscriber for control-unit transaction sampling.

Inheritance diagram for coverage:



Collaboration diagram for coverage:



Public Member Functions

- [uvm_component_utils](#) ([coverage](#)) [pkt tr](#)
Transaction currently being sampled by [cg_control_unit](#).
- covergroup [cg_control_unit](#) ()
Decode-space and PC-redirect coverage model.
- [new](#) (string name="coverage", [uvm_component](#) parent)
- virtual void [write](#) ([pkt t](#))
Samples the covergroup for one transaction and republishes the running coverage percentage.

8.3.1 Detailed Description

UVM coverage subscriber for control-unit transaction sampling.

Tracks opcode x funct3 x funct7[5] cross coverage (the decode space that selects [o_alu_ctrl_ID](#)/[o_imm_src_ID](#)/etc.), plus the branch/jump/zero cross that drives [o_pc_src_EX](#). Publishes the current percentage to the sequence layer through the configuration database under the key "cov_status".

8.3.2 Member Function Documentation

8.3.2.1 [cg_control_unit\(\)](#)

```
covergroup coverage::cg_control_unit () [inline]
```

Decode-space and PC-redirect coverage model.

One bin per RV32I base opcode this design implements (see [op_decoder.v](#)).

All 8 funct3 codes.

Both funct7[5] states (ADD/SUB and SRL/SRA discriminator).

Both states of the registered branch/jump/zero-flag inputs, to exercise every path through [o_pc_src_EX](#) = (zero && branch_EX) || jump_EX. Both states of the registered branch flag at Execute.

Both states of the registered jump flag at Execute.

Both states of the ALU zero/branch-comparison flag.

Crosses branch, jump and zero so every path to [o_pc_src_EX](#) is exercised.

Exercises every (opcode, funct3, funct7[5]) combination this design can see.

8.3.2.2 [new\(\)](#)

```
coverage::new (
    string name = "coverage",
    uvm_component parent) [inline]
```

8.3.2.3 uvm_component_utils()

```
coverage::uvm_component_utils (
    coverage )
```

Transaction currently being sampled by [cg_control_unit](#).

8.3.2.4 write()

```
virtual void coverage::write (
    pkt t) [inline], [virtual]
```

Samples the covergroup for one transaction and republishes the running coverage percentage.

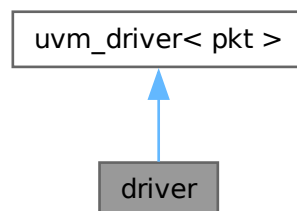
The documentation for this class was generated from the following file:

- [uvm/coverage.sv](#)

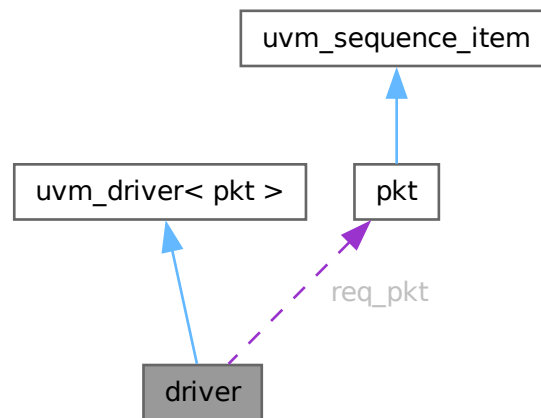
8.4 driver Class Reference

UVM driver for control-unit transactions.

Inheritance diagram for driver:



Collaboration diagram for driver:



Public Member Functions

- [uvm_component_utils](#) ([driver](#)) virtual s_interface.driver [vif](#)
Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").
- [new](#) (string name="driver", uvm_component parent)
- virtual void [build_phase](#) (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task [run_phase](#) (uvm_phase phase)
Waits for reset to be released, then drives sequence items forever.
- virtual task [drive_item](#) ([pkt](#) [pkt_item](#))
Drives one full instruction-field/EX-feedback set for one clock cycle.

Public Attributes

- [pkt req_pkt](#)
Sequence item currently being driven.

8.4.1 Detailed Description

UVM driver for control-unit transactions.

Converts each transaction item into the corresponding stimulus pulse on [s_interface](#). Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

8.4.2 Member Function Documentation

8.4.2.1 build_phase()

```
virtual void driver::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.4.2.2 drive_item()

```
virtual task driver::drive_item (  
    pkt pkt_item) [inline], [virtual]
```

Drives one full instruction-field/EX-feedback set for one clock cycle.

8.4.2.3 new()

```
driver::new (  
    string name = "driver",  
    uvm_component parent) [inline]
```

8.4.2.4 run_phase()

```
virtual task driver::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Waits for reset to be released, then drives sequence items forever.

8.4.2.5 uvm_component_utils()

```
driver::uvm_component_utils (  
    driver )
```

Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.4.3 Member Data Documentation

8.4.3.1 req_pkt

```
pkt driver::req_pkt
```

Sequence item currently being driven.

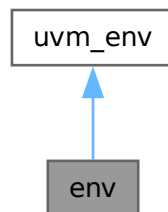
The documentation for this class was generated from the following file:

- [uvm/driver.sv](#)

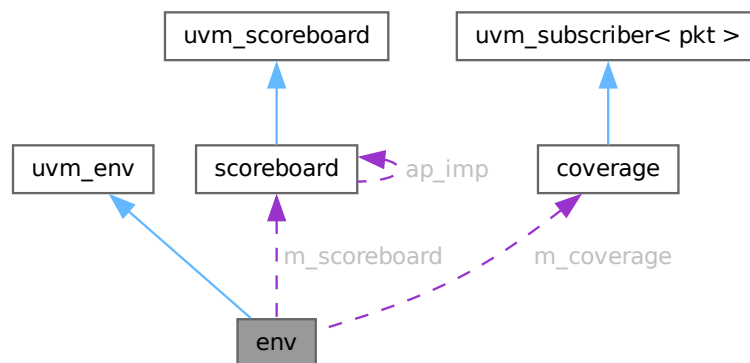
8.5 env Class Reference

Top-level UVM environment for the control-unit verification bench.

Inheritance diagram for env:



Collaboration diagram for env:



Public Member Functions

- `uvm_component_utils (env) agent m_agent`
Agent that drives and monitors the DUT.
- `new (string name="env", uvm_component parent=null)`
- `virtual void build_phase (uvm_phase phase)`
Creates the agent, scoreboard, and coverage collector.
- `virtual void connect_phase (uvm_phase phase)`
Connects the monitor's analysis port to both the scoreboard and coverage collector.

Public Attributes

- [scoreboard m_scoreboard](#)
Checks monitored transactions against the reference model.
- [coverage m_coverage](#)
Collects functional coverage of monitored transactions.

8.5.1 Detailed Description

Top-level UVM environment for the control-unit verification bench.

Instantiates the agent, scoreboard, and coverage collector and connects the monitored transactions to the checkers.

8.5.2 Member Function Documentation

8.5.2.1 build_phase()

```
virtual void env::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the agent, scoreboard, and coverage collector.

8.5.2.2 connect_phase()

```
virtual void env::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the monitor's analysis port to both the scoreboard and coverage collector.

8.5.2.3 new()

```
env::new (  
    string name = "env",  
    uvm_component parent = null) [inline]
```

8.5.2.4 uvm_component_utils()

```
env::uvm_component_utils (  
    env )
```

Agent that drives and monitors the DUT.

8.5.3 Member Data Documentation

8.5.3.1 m_coverage

```
coverage env::m_coverage
```

Collects functional coverage of monitored transactions.

8.5.3.2 m_scoreboard

`scoreboard env::m_scoreboard`

Checks monitored transactions against the reference model.

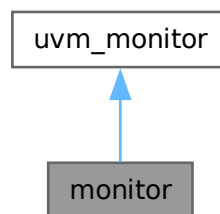
The documentation for this class was generated from the following file:

- [uvm/env.sv](#)

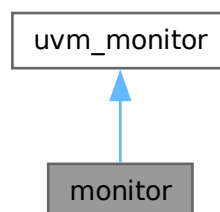
8.6 monitor Class Reference

UVM monitor for control-unit transaction observation.

Inheritance diagram for monitor:



Collaboration diagram for monitor:



Public Member Functions

- [uvm_component_utils](#) ([monitor](#)) virtual `s_interface.monitor vif`
Monitor modport of the DUT interface, obtained from the config DB (key "[vif](#)").
- [new](#) (string name="monitor", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task [run_phase](#) (uvm_phase phase)
Samples one control-unit transaction per iteration and publishes it.

Public Attributes

- `uvm_analysis_port< pkt > mon_analysis_port`
Publishes each observed transaction to the scoreboard and coverage collector.

8.6.1 Detailed Description

UVM monitor for control-unit transaction observation.

Samples the DUT-visible signals on each transfer and publishes the resulting transaction object to downstream analysis components. Requires the virtual interface to be set in the configuration database under the key "`vif`".

8.6.2 Member Function Documentation

8.6.2.1 build_phase()

```
virtual void monitor::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.6.2.2 new()

```
monitor::new (  
    string name = "monitor",  
    uvm_component parent = null) [inline]
```

8.6.2.3 run_phase()

```
virtual task monitor::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Samples one control-unit transaction per iteration and publishes it.

`control_unit_wrapper` is purely combinational, but its outputs only settle once the DUT's decode logic reacts to the new inputs, which is not guaranteed to have happened yet in the same simulation step that wakes this process on `enable`'s rising edge. A minimal `#1` delay lets that propagation finish before the outputs are sampled.

8.6.2.4 uvm_component_utils()

```
monitor::uvm_component_utils (  
    monitor )
```

Monitor modport of the DUT interface, obtained from the config DB (key "`vif`").

8.6.3 Member Data Documentation

8.6.3.1 mon_analysis_port

`uvm_analysis_port<pkt> monitor::mon_analysis_port`

Publishes each observed transaction to the scoreboard and coverage collector.

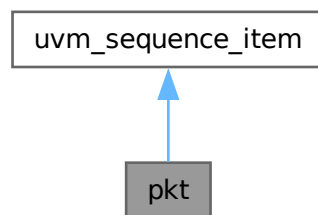
The documentation for this class was generated from the following file:

- `uvm/monitor.sv`

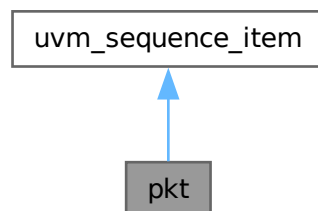
8.7 pkt Class Reference

Sequence item that models one control-unit decode (stimulus + expected response fields).

Inheritance diagram for pkt:



Collaboration diagram for pkt:



Public Member Functions

- `uvm_object_utils_begin (pkt) uvm_field_int(i_op`
- `UVM_DEFAULT uvm_field_int (i_funcnt_3, UVM_DEFAULT) uvm_field_int(i_funcnt_7_5`
- `UVM_DEFAULT UVM_DEFAULT uvm_field_int (i_branch_EX, UVM_DEFAULT) uvm_field_int(i_jump_EX`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (i_zero, UVM_DEFAULT) uvm_field_int(o_pc_src_EX`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_jump_ID, UVM_DEFAULT) uvm_field_int(o_branch_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_reg_write_ID, UVM_DEFAULT) uvm_field_int(o_result_src_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_mem_write_ID, UVM_DEFAULT) uvm_field_int(o_alu_ctrl_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_alu_src_ID, UVM_DEFAULT) uvm_field_int(o_addr_src_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_imm_src_ID, UVM_DEFAULT) uvm_field_int(o_fence_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_field_int (o_imm_src_ID, UVM_DEFAULT) uvm_field_int(o_fence_ID`
- `UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT uvm_object_utils_end constraint op_range ()`
- `constraint branch_funcnt3_valid ()`
- `new (string name="pkt")`

Public Attributes

- rand logic < 4:0 > **i_op**
RV32I base opcode, instr[6:2].
- rand logic < 2:0 > **i_funct_3**
instr[14:12].
- rand logic **i_funct_7_5**
instr[30].
- rand logic **i_branch_EX**
Registered o_branch_ID at Execute.
- rand logic **i_jump_EX**
Registered o_jump_ID at Execute.
- rand logic **i_zero**
ALU branch-comparison result.
- logic **o_pc_src_EX**
Sampled by the monitor.
- logic **o_jump_ID**
Sampled by the monitor.
- logic **o_branch_ID**
Sampled by the monitor.
- logic **o_reg_write_ID**
Sampled by the monitor.
- logic < 1:0 > **o_result_src_ID**
Sampled by the monitor.
- logic **o_mem_write_ID**
Sampled by the monitor.
- logic < 4:0 > **o_alu_ctrl_ID**
Sampled by the monitor.
- logic **o_alu_src_ID**
Sampled by the monitor.

- logic [o_addr_src_ID](#)
Sampled by the monitor.
- logic< 2:0 > [o_imm_src_ID](#)
Sampled by the monitor.
- logic [o_fence_ID](#)
Sampled by the monitor.

8.7.1 Detailed Description

Sequence item that models one control-unit decode (stimulus + expected response fields).

Carries the fields randomized by the sequence/driver, and the result fields filled in by the monitor for the scoreboard to check against the golden reference model.

8.7.2 Member Function Documentation

8.7.2.1 `branch_funct3_valid()`

```
constraint pkt::branch_funct3_valid () [inline]
```

RV32I only defines 6 branch conditions (BEQ/BNE/BLT/BGE/BLTU/BGEU); funct3 == 3'b010/3'b011 are reserved and don't correspond to a real branch.

8.7.2.2 `new()`

```
pkt::new (
    string name = "pkt") [inline]
```

8.7.2.3 `op_range()`

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_↔
DEFAULT UVM_DEFAULT uvm_object_utils_end constraint pkt::op_range () [inline]
```

Restricts the randomized opcode to the opcodes whose o_alu_ctrl_ID/o_alu_src_ID have a real, checkable answer (i.e. the instruction writes a register or touches memory). FENCE and SYSTEM (ECALL/EBREAK) are excluded: they never use the ALU result, so there is no defined expected value for those two fields.

8.7.2.4 `uvm_field_int()` [1/8]

```
UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_branch_EX ,
    UVM_DEFAULT )
```

8.7.2.5 `uvm_field_int()` [2/8]

```
UVM_DEFAULT pkt::uvm_field_int (
    i_funct_3 ,
    UVM_DEFAULT )
```

8.7.2.6 uvm_field_int() [3/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_zero ,
    UVM_DEFAULT )
```

8.7.2.7 uvm_field_int() [4/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm↔
_field_int (
    o_alu_src_ID ,
    UVM_DEFAULT )
```

8.7.2.8 uvm_field_int() [5/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM↔
DEFAULT pkt::uvm_field_int (
    o_imm_src_ID ,
    UVM_DEFAULT )
```

8.7.2.9 uvm_field_int() [6/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    o_jump_ID ,
    UVM_DEFAULT )
```

8.7.2.10 uvm_field_int() [7/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    o_mem_write_ID ,
    UVM_DEFAULT )
```

8.7.2.11 uvm_field_int() [8/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    o_reg_write_ID ,
    UVM_DEFAULT )
```

8.7.2.12 uvm_object_utils_begin()

```
pkt::uvm_object_utils_begin (
    pkt )
```

8.7.3 Member Data Documentation

8.7.3.1 i_branch_EX

```
rand logic pkt::i_branch_EX
```

Registered o_branch_ID at Execute.

8.7.3.2 i_funct_3

```
rand logic<2:0> pkt::i_funct_3
```

instr[14:12].

8.7.3.3 i_funct_7_5

```
rand logic pkt::i_funct_7_5
```

instr[30].

8.7.3.4 i_jump_EX

```
rand logic pkt::i_jump_EX
```

Registered o_jump_ID at Execute.

8.7.3.5 i_op

```
rand logic<4:0> pkt::i_op
```

RV32I base opcode, instr[6:2].

8.7.3.6 i_zero

```
rand logic pkt::i_zero
```

ALU branch-comparison result.

8.7.3.7 o_addr_src_ID

```
logic pkt::o_addr_src_ID
```

Sampled by the monitor.

8.7.3.8 o_alu_ctrl_ID

```
logic<4:0> pkt::o_alu_ctrl_ID
```

Sampled by the monitor.

8.7.3.9 o_alu_src_ID

```
logic pkt::o_alu_src_ID
```

Sampled by the monitor.

8.7.3.10 o_branch_ID

```
logic pkt::o_branch_ID
```

Sampled by the monitor.

8.7.3.11 o_fence_ID

```
logic pkt::o_fence_ID
```

Sampled by the monitor.

8.7.3.12 o_imm_src_ID

```
logic<2:0> pkt::o_imm_src_ID
```

Sampled by the monitor.

8.7.3.13 o_jump_ID

```
logic pkt::o_jump_ID
```

Sampled by the monitor.

8.7.3.14 o_mem_write_ID

```
logic pkt::o_mem_write_ID
```

Sampled by the monitor.

8.7.3.15 o_pc_src_EX

```
logic pkt::o_pc_src_EX
```

Sampled by the monitor.

8.7.3.16 o_reg_write_ID

```
logic pkt::o_reg_write_ID
```

Sampled by the monitor.

8.7.3.17 o_result_src_ID

```
logic<1:0> pkt::o_result_src_ID
```

Sampled by the monitor.

The documentation for this class was generated from the following file:

- [uvm/pkt.sv](#)

8.8 s_interface Interface Reference

Control-unit stimulus/response interface shared by the DUT and the UVM environment.

8.8.1 Detailed Description

Control-unit stimulus/response interface shared by the DUT and the UVM environment.

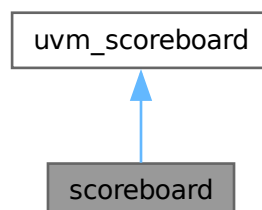
The documentation for this interface was generated from the following file:

- [tb/s_interface.sv](#)

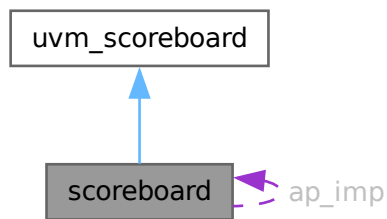
8.9 scoreboard Class Reference

UVM scoreboard for control-unit functional checking.

Inheritance diagram for scoreboard:



Collaboration diagram for scoreboard:



Public Member Functions

- `uvm_component_utils (scoreboard) uvm_analysis_imp< pkt`
- `new` (string name="scoreboard", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Creates the analysis imp that receives transactions from the monitor.
- virtual void `write` (pkt data)
Analysis-port callback: checks one monitored transaction against `control_unit_golden`.
- virtual void `check_phase` (uvm_phase phase)
Fails the test if any mismatch was recorded by `write()`.
- virtual task `run_phase` (uvm_phase phase)
Run phase; unused, checking happens in `write()` and `check_phase()`.

Public Attributes

- `scoreboard ap_imp`
Receives transactions from the monitor's analysis port.
- int `num_errors` = 0
Number of mismatches found so far.

8.9.1 Detailed Description

UVM scoreboard for control-unit functional checking.

Compares monitored control-unit inputs/outputs with the DPI-C reference model and reports mismatches as test failures.

8.9.2 Member Function Documentation

8.9.2.1 build_phase()

```
virtual void scoreboard::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Creates the analysis imp that receives transactions from the monitor.

8.9.2.2 check_phase()

```
virtual void scoreboard::check_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fails the test if any mismatch was recorded by `write()`.

8.9.2.3 new()

```
scoreboard::new (  
    string name = "scoreboard",  
    uvm_component parent = null) [inline]
```

8.9.2.4 run_phase()

```
virtual task scoreboard::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Run phase; unused, checking happens in `write()` and `check_phase()`.

8.9.2.5 uvm_component_utils()

```
scoreboard::uvm_component_utils (  
    scoreboard )
```

8.9.2.6 write()

```
virtual void scoreboard::write (  
    pkt data) [inline], [virtual]
```

Analysis-port callback: checks one monitored transaction against `control_unit_golden`.

8.9.3 Member Data Documentation

8.9.3.1 ap_imp

```
scoreboard scoreboard::ap_imp
```

Receives transactions from the monitor's analysis port.

8.9.3.2 num_errors

```
int scoreboard::num_errors = 0
```

Number of mismatches found so far.

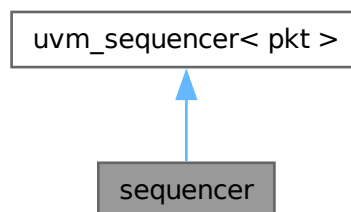
The documentation for this class was generated from the following file:

- [uvm/scoreboard.sv](#)

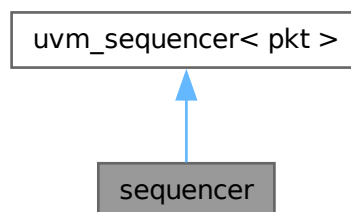
8.10 sequencer Class Reference

UVM sequencer for control-unit transactions.

Inheritance diagram for sequencer:



Collaboration diagram for sequencer:



Public Member Functions

- [uvm_component_utils](#) ([sequencer](#)) new(string name

Public Attributes

- [uvm_component](#) [parent](#)

8.10.1 Detailed Description

UVM sequencer for control-unit transactions.

Supplies randomized sequence items to the driver during the test run.

8.10.2 Member Function Documentation

8.10.2.1 `uvm_component_utils()`

```
sequencer::uvm_component_utils (
    sequencer ) [new]
```

8.10.3 Member Data Documentation

8.10.3.1 `parent`

```
uvm_component sequencer::parent
```

Initial value:

```
{
    super.new (name, parent)
}
```

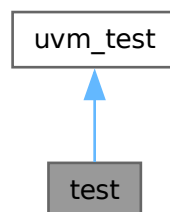
The documentation for this class was generated from the following file:

- [uvm/sequencer.sv](#)

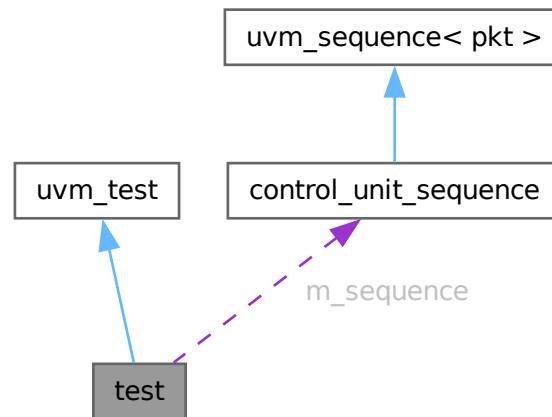
8.11 test Class Reference

Top-level UVM test for the control-unit verification bench.

Inheritance diagram for test:



Collaboration diagram for test:



Public Member Functions

- `uvm_component_utils (test) env m_env`
Verification environment under test.
- `new` (string name="test", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Builds the verification environment.
- virtual void `end_of_elaboration_phase` (uvm_phase phase)
Prints the component topology once elaboration is complete.
- virtual task `run_phase` (uvm_phase phase)
Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

Public Attributes

- `control_unit_sequence m_sequence`
Main stimulus sequence, started on the agent's sequencer.

8.11.1 Detailed Description

Top-level UVM test for the control-unit verification bench.

Builds the environment, starts the main sequence, and drives the full verification flow from the test root.

8.11.2 Member Function Documentation

8.11.2.1 build_phase()

```
virtual void test::build_phase (
    uvm_phase phase) [inline], [virtual]
```

Builds the verification environment.

8.11.2.2 end_of_elaboration_phase()

```
virtual void test::end_of_elaboration_phase (
    uvm_phase phase) [inline], [virtual]
```

Prints the component topology once elaboration is complete.

8.11.2.3 new()

```
test::new (
    string name = "test",
    uvm_component parent = null) [inline]
```

8.11.2.4 run_phase()

```
virtual task test::run_phase (
    uvm_phase phase) [inline], [virtual]
```

Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

8.11.2.5 uvm_component_utils()

```
test::uvm_component_utils (
    test )
```

Verification environment under test.

8.11.3 Member Data Documentation

8.11.3.1 m_sequence

```
control_unit_sequence test::m_sequence
```

Main stimulus sequence, started on the agent's sequencer.

The documentation for this class was generated from the following file:

- [uvm/test.sv](#)

Chapter 9

File Documentation

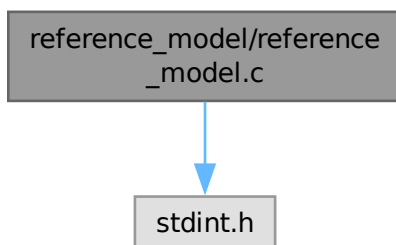
9.1 filter/README.md File Reference

9.2 reference_model/reference_model.c File Reference

Golden reference model for the control unit, called from the UVM scoreboard via DPI-C.

```
#include <stdint.h>
```

Include dependency graph for reference_model.c:



Macros

- `#define OP_LOAD 0x00u`
- `#define OP_IMM 0x04u`
- `#define OP_AUIPC 0x05u`
- `#define OP_STORE 0x08u`
- `#define OP_REG 0x0Cu`
- `#define OP_LUI 0x0Du`
- `#define OP_BRANCH 0x18u`
- `#define OP_JALR 0x19u`
- `#define OP_JAL 0x1Bu`

- #define `ALU_AND` 0u
- #define `ALU_OR` 1u
- #define `ALU_XOR` 2u
- #define `ALU_ADD` 3u
- #define `ALU_SUB` 4u
- #define `ALU_SLL` 5u
- #define `ALU_SRL` 6u
- #define `ALU_SLT` 7u
- #define `ALU_SLTU` 8u
- #define `ALU_SRA` 9u
- #define `ALU_BEQ` 10u
- #define `ALU_BNE` 11u
- #define `ALU_BLT` 12u
- #define `ALU_BLTU` 13u
- #define `ALU_BGE` 14u
- #define `ALU_BGEU` 15u
- #define `ALU_LUI` 16u
- #define `IMM_I` 0u
- #define `IMM_S` 1u
- #define `IMM_B` 2u
- #define `IMM_J` 3u
- #define `IMM_U` 4u
- #define `RESULT_ALU` 0u
- #define `RESULT_PC4` 1u
- #define `RESULT_MEM` 2u
- #define `RESULT_PCIMM` 3u

Functions

- void `control_unit_golden` (unsigned int op, unsigned int funct3, unsigned char funct7_5, unsigned char branch_ex, unsigned char jump_ex, unsigned char zero, unsigned char *exp_jump_id, unsigned char *exp_branch_id, unsigned char *exp_reg_write_id, unsigned int *exp_result_src_id, unsigned char *exp_mem_write_id, unsigned int *exp_alu_ctrl_id, unsigned char *exp_alu_src_id, unsigned char *exp_addr_src_id, unsigned int *exp_imm_src_id, unsigned char *exp_fence_id, unsigned char *exp_pc_src_ex)

Computes the expected control-unit outputs for one instruction.

9.2.1 Detailed Description

Golden reference model for the control unit, called from the UVM scoreboard via DPI-C.

Derives every control signal directly from the RISC-V base ISA instruction formats (RV32I opcode map, instr[6:2]) and their funct3/ funct7[5] encoding, independently of the RTL decoders under test. FENCE and SYSTEM (ECALL/EBREAK) are not modeled: they never use the ALU result, so o_alu_ctrl_ID/o_alu_src_ID have no defined expected value for them; `pkt.sv` excludes those two opcodes from randomization.

9.2.2 Macro Definition Documentation

9.2.2.1 ALU_ADD

```
#define ALU_ADD 3u
```

9.2.2.2 ALU_AND

```
#define ALU_AND 0u
```

9.2.2.3 ALU_BEQ

```
#define ALU_BEQ 10u
```

9.2.2.4 ALU_BGE

```
#define ALU_BGE 14u
```

9.2.2.5 ALU_BGEU

```
#define ALU_BGEU 15u
```

9.2.2.6 ALU_BLT

```
#define ALU_BLT 12u
```

9.2.2.7 ALU_BLTU

```
#define ALU_BLTU 13u
```

9.2.2.8 ALU_BNE

```
#define ALU_BNE 11u
```

9.2.2.9 ALU_LUI

```
#define ALU_LUI 16u
```

9.2.2.10 ALU_OR

```
#define ALU_OR 1u
```

9.2.2.11 ALU_SLL

```
#define ALU_SLL 5u
```

9.2.2.12 ALU_SLT

```
#define ALU_SLT 7u
```

9.2.2.13 ALU_SLTU

```
#define ALU_SLTU 8u
```

9.2.2.14 ALU_SRA

```
#define ALU_SRA 9u
```

9.2.2.15 ALU_SRL

```
#define ALU_SRL 6u
```

9.2.2.16 ALU_SUB

```
#define ALU_SUB 4u
```

9.2.2.17 ALU_XOR

```
#define ALU_XOR 2u
```

9.2.2.18 IMM_B

```
#define IMM_B 2u
```

9.2.2.19 IMM_I

```
#define IMM_I 0u
```

9.2.2.20 IMM_J

```
#define IMM_J 3u
```

9.2.2.21 IMM_S

```
#define IMM_S 1u
```

9.2.2.22 IMM_U

```
#define IMM_U 4u
```

9.2.2.23 OP_AUIPC

```
#define OP_AUIPC 0x05u
```

9.2.2.24 OP_BRANCH

```
#define OP_BRANCH 0x18u
```

9.2.2.25 OP_IMM

```
#define OP_IMM 0x04u
```

9.2.2.26 OP_JAL

```
#define OP_JAL 0x1Bu
```

9.2.2.27 OP_JALR

```
#define OP_JALR 0x19u
```

9.2.2.28 OP_LOAD

```
#define OP_LOAD 0x00u
```

9.2.2.29 OP_LUI

```
#define OP_LUI 0x0Du
```

9.2.2.30 OP_REG

```
#define OP_REG 0x0Cu
```

9.2.2.31 OP_STORE

```
#define OP_STORE 0x08u
```

9.2.2.32 RESULT_ALU

```
#define RESULT_ALU 0u
```

9.2.2.33 RESULT_MEM

```
#define RESULT_MEM 2u
```

9.2.2.34 RESULT_PC4

```
#define RESULT_PC4 1u
```

9.2.2.35 RESULT_PCIMM

```
#define RESULT_PCIMM 3u
```

9.2.3 Function Documentation

9.2.3.1 control_unit_golden()

```
void control_unit_golden (
    unsigned int op,
    unsigned int funct3,
    unsigned char funct7_5,
    unsigned char branch_ex,
    unsigned char jump_ex,
    unsigned char zero,
    unsigned char * exp_jump_id,
    unsigned char * exp_branch_id,
    unsigned char * exp_reg_write_id,
    unsigned int * exp_result_src_id,
    unsigned char * exp_mem_write_id,
    unsigned int * exp_alu_ctrl_id,
    unsigned char * exp_alu_src_id,
    unsigned char * exp_addr_src_id,
    unsigned int * exp_imm_src_id,
    unsigned char * exp_fence_id,
    unsigned char * exp_pc_src_ex)
```

Computes the expected control-unit outputs for one instruction.

Parameters

<i>op</i>	instr[6:2], the RV32I base opcode.
<i>funct3</i>	instr[14:12].

<i>funct7_5</i>	instr[30].
<i>branch_ex</i>	1 if the instruction at Execute is a branch (o_branch_ID, registered).
<i>jump_ex</i>	1 if the instruction at Execute is a jump (o_jump_ID, registered).
<i>zero</i>	1 if the ALU's branch comparison result is true.
<i>exp_jump_id</i>	Output: expected o_jump_ID.
<i>exp_branch_id</i>	Output: expected o_branch_ID.
<i>exp_reg_write_id</i>	Output: expected o_reg_write_ID.
<i>exp_result_src_id</i>	Output: expected o_result_src_ID (see RESULT_* above).
<i>exp_mem_write_id</i>	Output: expected o_mem_write_ID.
<i>exp_alu_ctrl_id</i>	Output: expected o_alu_ctrl_ID (see ALU_* above).
<i>exp_alu_src_id</i>	Output: expected o_alu_src_ID.
<i>exp_addr_src_id</i>	Output: expected o_addr_src_ID.
<i>exp_imm_src_id</i>	Output: expected o_imm_src_ID (see IMM_* above).
<i>exp_fence_id</i>	Output: expected o_fence_ID.
<i>exp_pc_src_ex</i>	Output: expected o_pc_src_EX.

9.3 rtl/alu_decoder.v File Reference

Variables

- input logic[2:0] [i_alu_op](#)
- input logic[2:0] [i_funct_3](#)
- input logic [i_funct_7_5](#)
- output logic[4:0] [o_alu_ctrl_ID](#)
- localparam logic[2:0] [OP_LUI](#) = 3'b000
- localparam logic[2:0] [OP_ARITH](#) = 3'b001
- localparam logic[2:0] [OP_ADD_SUB](#) = 3'b010
- localparam logic[2:0] [OP_BRANCH](#) = 3'b011
- localparam logic[2:0] [OP_ADD](#) = 3'b100
- localparam logic[4:0] [AND](#) = 5'b00000
- localparam logic[4:0] [OR](#) = 5'b00001
- localparam logic[4:0] [XOR](#) = 5'b00010
- localparam logic[4:0] [ADD](#) = 5'b00011
- localparam logic[4:0] [SUB](#) = 5'b00100
- localparam logic[4:0] [SLL](#) = 5'b00101
- localparam logic[4:0] [SRL](#) = 5'b00110
- localparam logic[4:0] [SLT](#) = 5'b00111
- localparam logic[4:0] [SLTU](#) = 5'b01000

- localparam logic[4:0] **SRA** = 5'b01001
- localparam logic[4:0] **BEQ** = 5'b01010
- localparam logic[4:0] **BNE** = 5'b01011
- localparam logic[4:0] **BLT** = 5'b01100
- localparam logic[4:0] **BLTU** = 5'b01101
- localparam logic[4:0] **BGE** = 5'b01110
- localparam logic[4:0] **BGEU** = 5'b01111
- localparam logic[4:0] **LUI** = 5'b10000
- localparam logic[4:0] **AUIPC** = 5'b10001
- localparam logic[4:0] **FENCE** = 5'b10010
- localparam logic[4:0] **ECALL** = 5'b10011
- localparam logic[4:0] **EBREAK** = 5'b10100

9.3.1 Variable Documentation

9.3.1.1 ADD

```
localparam logic [4:0] ADD = 5'b00011
```

9.3.1.2 AND

```
localparam logic [4:0] AND = 5'b00000
```

9.3.1.3 AUIPC

```
localparam logic [4:0] AUIPC = 5'b10001
```

9.3.1.4 BEQ

```
localparam logic [4:0] BEQ = 5'b01010
```

9.3.1.5 BGE

```
localparam logic [4:0] BGE = 5'b01110
```

9.3.1.6 BGEU

```
localparam logic [4:0] BGEU = 5'b01111
```

9.3.1.7 BLT

```
localparam logic [4:0] BLT = 5'b01100
```

9.3.1.8 BLTU

```
localparam logic [4:0] BLTU = 5'b01101
```

9.3.1.9 BNE

```
localparam logic [4:0] BNE = 5'b01011
```

9.3.1.10 EBREAK

```
localparam logic [4:0] EBREAK = 5'b10100
```

9.3.1.11 ECALL

```
localparam logic [4:0] ECALL = 5'b10011
```

9.3.1.12 FENCE

```
localparam logic [4:0] FENCE = 5'b10010
```

9.3.1.13 i_alu_op

```
input logic [2:0] i_alu_op
```

9.3.1.14 i_funct_3

```
input logic [2:0] i_funct_3
```

9.3.1.15 i_funct_7_5

```
input logic i_funct_7_5
```

9.3.1.16 LUI

```
localparam logic [4:0] LUI = 5'b10000
```

9.3.1.17 o_alu_ctrl_ID

assign o_alu_ctrl_ID

Initial value:

```

=
((i_alu_op == OP_ADD_SUB) & (i_funct_3 == 3'b000) & (i_funct_7_5 == 1'b0)) ? ADD :
((i_alu_op == OP_ADD_SUB) & (i_funct_3 == 3'b000) & (i_funct_7_5 == 1'b1)) ? SUB :
((i_alu_op == OP_ADD_SUB)) ? ADD :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b000)) ? ADD :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b001)) ? SLL :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b010)) ? SLT :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b011)) ? SLTU :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b100)) ? XOR :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b101) & (i_funct_7_5 == 1'b0)) ? SRL :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b101) & (i_funct_7_5 == 1'b1)) ? SRA :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b110)) ? OR :
((i_alu_op == OP_ARITH) & (i_funct_3 == 3'b111)) ? AND :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b000)) ? BEQ :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b001)) ? BNE :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b100)) ? BLT :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b110)) ? BLTU :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b101)) ? BGE :
((i_alu_op == OP_BRANCH) & (i_funct_3 == 3'b111)) ? BGEU :
((i_alu_op == OP_LUI)) ? LUI :
((i_alu_op == OP_ADD)) ? ADD :
5'b0zzzz

```

9.3.1.18 OP_ADD

localparam logic [2:0] OP_ADD = 3'b100

9.3.1.19 OP_ADD_SUB

localparam logic [2:0] OP_ADD_SUB = 3'b010

9.3.1.20 OP_ARITH

localparam logic [2:0] OP_ARITH = 3'b001

9.3.1.21 OP_BRANCH

localparam logic [2:0] OP_BRANCH = 3'b011

9.3.1.22 OP_LUI

localparam logic [2:0] OP_LUI = 3'b000

9.3.1.23 OR

localparam logic [4:0] OR = 5'b00001

9.3.1.24 SLL

```
localparam logic [4:0] SLL = 5'b00101
```

9.3.1.25 SLT

```
localparam logic [4:0] SLT = 5'b00111
```

9.3.1.26 SLTU

```
localparam logic [4:0] SLTU = 5'b01000
```

9.3.1.27 SRA

```
localparam logic [4:0] SRA = 5'b01001
```

9.3.1.28 SRL

```
localparam logic [4:0] SRL = 5'b00110
```

9.3.1.29 SUB

```
localparam logic [4:0] SUB = 5'b00100
```

9.3.1.30 XOR

```
localparam logic [4:0] XOR = 5'b00010
```

9.4 rtl/control_unit.v File Reference

Functions

- [op_decoder U_OP_DECODER](#) (.i_op(i_op),.i_funct_3(i_funct_3),.i_funct_7_5(i_funct_7_5),.o_jump_ID(o_jump_ID),.o_branch
- [alu_decoder U_ALU_DECODER](#) (.i_alu_op(alu_op),.i_funct_3(i_funct_3),.i_funct_7_5(i_funct_7_5),.o_alu_ctrl_ID(o_alu_ctrl_I

Variables

- input logic[4:0] `i_op`
- input logic[2:0] `i_funct_3`
- input logic `i_funct_7_5`
- input logic `i_zero`
- input logic `i_branch_EX`
- input logic `i_jump_EX`
- output logic `o_pc_src_EX` = $((i_zero \&\& i_branch_EX) \parallel i_jump_EX) ? 1'b1 : 1'b0$
- output logic `o_jump_ID` = $(i_op == J_TYPE_JAL \mid i_op == I_TYPE_JALR) ? 1'b1 : 1'b0$
- output logic `o_branch_ID` = $(i_op == B_TYPE_BRANCH) ? 1'b1 : 1'b0$
- output logic `o_reg_write_ID`
- output logic[1:0] `o_result_src_ID`
- output logic `o_mem_write_ID` = $(i_op == S_TYPE) ? 1'b1 : 1'b0$
- output logic[4:0] `o_alu_ctrl_ID`
- output logic `o_alu_src_ID`
- output logic `o_addr_src_ID` = $(i_op == I_TYPE_JALR) ? 1'b1 : 1'b0$
- output logic[2:0] `o_imm_src_ID`
- output logic `o_fence_ID` = $(i_op == I_TYPE_FENCE) ? 1'b1 : 1'b0$
- logic[2:0] `alu_op`

9.4.1 Function Documentation

9.4.1.1 U_ALU_DECODER()

```
alu_decoder U_ALU_DECODER (
    . i_alu_opalu_op,
    . i_funct_3i_funct_3,
    . i_funct_7_5i_funct_7_5,
    . o_alu_ctrl_IDo_alu_ctrl_ID)
```

9.4.1.2 U_OP_DECODER()

```
op_decoder U_OP_DECODER (
    . i_opi_op,
    . i_funct_3i_funct_3,
    . i_funct_7_5i_funct_7_5,
    . o_jump_IDo_jump_ID,
    . o_branch_IDo_branch_ID,
    . o_reg_write_IDo_reg_write_ID,
    . o_result_src_IDo_result_src_ID,
    . o_mem_write_IDo_mem_write_ID,
    . o_alu_src_IDo_alu_src_ID,
    . o_imm_src_IDo_imm_src_ID,
    . o_alu_opalu_op,
    . o_addr_src_IDo_addr_src_ID,
    . o_fence_IDo_fence_ID)
```

9.4.2 Variable Documentation

9.4.2.1 alu_op

```
logic [2:0] alu_op
```

9.4.2.2 i_branch_EX

```
input logic i_branch_EX
```

9.4.2.3 i_funct_3

```
input logic [2:0] i_funct_3
```

9.4.2.4 i_funct_7_5

```
input logic i_funct_7_5
```

9.4.2.5 i_jump_EX

```
input logic i_jump_EX
```

9.4.2.6 i_op

```
input logic [4:0] i_op
```

9.4.2.7 i_zero

```
input logic i_zero
```

9.4.2.8 o_addr_src_ID

```
assign o_addr_src_ID = (i_op == I_TYPE_JALR) ? 1'b1 : 1'b0
```

9.4.2.9 o_alu_ctrl_ID

```
output logic [4:0] o_alu_ctrl_ID
```

9.4.2.10 o_alu_src_ID

```
assign o_alu_src_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI | i_op == U_TYPE_AUIPC | i_op == I_TYPE_JALR |  
   i_op == I_TYPE_LOAD | i_op == S_TYPE | i_op == I_TYPE_ARITH | i_op == I_TYPE_SYSTEM)  
  ? 1'b1 : 1'b0
```

9.4.2.11 o_branch_ID

```
assign o_branch_ID = (i_op == B_TYPE_BRANCH) ? 1'b1 : 1'b0
```

9.4.2.12 o_fence_ID

```
assign o_fence_ID = (i_op == I_TYPE_FENCE) ? 1'b1 : 1'b0
```

9.4.2.13 o_imm_src_ID

```
assign o_imm_src_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI) ?  
  U_type : (i_op == U_TYPE_AUIPC) ? U_type : (i_op == J_TYPE_JAL) ? J_type :  
  (i_op == B_TYPE_BRANCH) ? B_type : (i_op == S_TYPE) ? S_type : I_type
```

9.4.2.14 o_jump_ID

```
assign o_jump_ID = (i_op == J_TYPE_JAL | i_op == I_TYPE_JALR) ? 1'b1 : 1'b0
```

9.4.2.15 o_mem_write_ID

```
assign o_mem_write_ID = (i_op == S_TYPE) ? 1'b1 : 1'b0
```

9.4.2.16 o_pc_src_EX

```
assign o_pc_src_EX = ((i_zero && i_branch_EX) || i_jump_EX) ? 1'b1 : 1'b0
```

9.4.2.17 o_reg_write_ID

```
assign o_reg_write_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI | i_op == U_TYPE_AUIPC | i_op == J_TYPE_JAL |  
  i_op == I_TYPE_JALR | i_op == I_TYPE_LOAD | i_op == I_TYPE_ARITH |  
  i_op == R_TYPE) ? 1'b1 : 1'b0
```

9.4.2.18 o_result_src_ID

```
assign o_result_src_ID
```

Initial value:

```
=  
(i_op == R_TYPE) ? 2'b00 :  
(i_op == I_TYPE_LOAD) ? 2'b10 :  
(i_op == I_TYPE_ARITH) ? 2'b00 :  
(i_op == I_TYPE_JALR) ? 2'b01 :  
(i_op == J_TYPE_JAL) ? 2'b01 :  
(i_op == U_TYPE_LUI) ? 2'b00 :  
(i_op == U_TYPE_AUIPC) ? 2'b11 :  
2'bxx
```

9.5 rtl/control_unit_wrapper.sv File Reference

Thin combinational wrapper around [control_unit](#), used as the DUT.

9.5.1 Detailed Description

Thin combinational wrapper around [control_unit](#), used as the DUT.

[control_unit](#) (and the [op_decoder/alu_decoder](#) it instantiates) has no internal state, so the wrapper simply wires the `s_interface.dut` modport straight into it, with no clock/reset logic.

9.6 rtl/op_decoder.v File Reference

Variables

- input logic[4:0] [i_op](#)
- input logic[2:0] [i_funct_3](#)
- input logic [i_funct_7_5](#)
- output logic [o_jump_ID](#) = ([i_op](#) == [J_TYPE_JAL](#) | [i_op](#) == [I_TYPE_JALR](#)) ? 1'b1 : 1'b0
- output logic [o_branch_ID](#) = ([i_op](#) == [B_TYPE_BRANCH](#)) ? 1'b1 : 1'b0
- output logic [o_reg_write_ID](#)
- output logic[1:0] [o_result_src_ID](#)
- output logic [o_mem_write_ID](#) = ([i_op](#) == [S_TYPE](#)) ? 1'b1 : 1'b0
- output logic [o_alu_src_ID](#)
- output logic[2:0] [o_imm_src_ID](#)
- output logic[2:0] [o_alu_op](#)
- output logic [o_addr_src_ID](#) = ([i_op](#) == [I_TYPE_JALR](#)) ? 1'b1 : 1'b0
- output logic [o_fence_ID](#) = ([i_op](#) == [I_TYPE_FENCE](#)) ? 1'b1 : 1'b0
- localparam logic[2:0] [OP_LUI](#) = 3'b000
- localparam logic[2:0] [OP_ARITH](#) = 3'b001
- localparam logic[2:0] [OP_ADD_SUB](#) = 3'b010
- localparam logic[2:0] [OP_BRANCH](#) = 3'b011
- localparam logic[2:0] [OP_ADD](#) = 3'b100
- localparam logic[2:0] [I_type](#) = 3'b000
- localparam logic[2:0] [S_type](#) = 3'b001
- localparam logic[2:0] [B_type](#) = 3'b010
- localparam logic[2:0] [J_type](#) = 3'b011
- localparam logic[2:0] [U_type](#) = 3'b100
- localparam logic[4:0] [R_TYPE](#) = 5'b01100
- localparam logic[4:0] [I_TYPE_LOAD](#) = 5'b00000
- localparam logic[4:0] [I_TYPE_ARITH](#) = 5'b00100
- localparam logic[4:0] [I_TYPE_JALR](#) = 5'b11001
- localparam logic[4:0] [J_TYPE_JAL](#) = 5'b11011
- localparam logic[4:0] [U_TYPE_LUI](#) = 5'b01101
- localparam logic[4:0] [U_TYPE_AUIPC](#) = 5'b00101
- localparam logic[4:0] [S_TYPE](#) = 5'b01000
- localparam logic[4:0] [B_TYPE_BRANCH](#) = 5'b11000
- localparam logic[4:0] [I_TYPE_FENCE](#) = 5'b00011
- localparam logic[4:0] [I_TYPE_SYSTEM](#) = 5'b11100

9.6.1 Variable Documentation

9.6.1.1 B_type

```
localparam logic [2:0] B_type = 3'b010
```

9.6.1.2 B_TYPE_BRANCH

```
localparam logic [4:0] B_TYPE_BRANCH = 5'b11000
```

9.6.1.3 i_funct_3

```
input logic [2:0] i_funct_3
```

9.6.1.4 i_funct_7_5

```
input logic i_funct_7_5
```

9.6.1.5 i_op

```
input logic [4:0] i_op
```

9.6.1.6 I_type

```
localparam logic [2:0] I_type = 3'b000
```

9.6.1.7 I_TYPE_ARITH

```
localparam logic [4:0] I_TYPE_ARITH = 5'b00100
```

9.6.1.8 I_TYPE_FENCE

```
localparam logic [4:0] I_TYPE_FENCE = 5'b00011
```

9.6.1.9 I_TYPE_JALR

```
localparam logic [4:0] I_TYPE_JALR = 5'b11001
```

9.6.1.10 I_TYPE_LOAD

```
localparam logic [4:0] I_TYPE_LOAD = 5'b00000
```

9.6.1.11 I_TYPE_SYSTEM

```
localparam logic [4:0] I_TYPE_SYSTEM = 5'b11100
```

9.6.1.12 J_type

```
localparam logic [2:0] J_type = 3'b011
```

9.6.1.13 J_TYPE_JAL

```
localparam logic [4:0] J_TYPE_JAL = 5'b11011
```

9.6.1.14 o_addr_src_ID

```
assign o_addr_src_ID = (i_op == I_TYPE_JALR) ? 1'b1 : 1'b0
```

9.6.1.15 o_alu_op

```
assign o_alu_op
```

9.6.1.16 o_alu_src_ID

```
assign o_alu_src_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI | i_op == U_TYPE_AUIPC | i_op == I_TYPE_JALR |  
   i_op == I_TYPE_LOAD | i_op == S_TYPE | i_op == I_TYPE_ARITH | i_op == I_TYPE_SYSTEM)  
  ? 1'b1 : 1'b0
```

9.6.1.17 o_branch_ID

```
assign o_branch_ID = (i_op == B_TYPE_BRANCH) ? 1'b1 : 1'b0
```

9.6.1.18 o_fence_ID

```
assign o_fence_ID = (i_op == I_TYPE_FENCE) ? 1'b1 : 1'b0
```

9.6.1.19 o_imm_src_ID

```
assign o_imm_src_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI) ?  
   U_type : (i_op == U_TYPE_AUIPC) ? U_type : (i_op == J_TYPE_JAL) ? J_type :  
   (i_op == B_TYPE_BRANCH) ? B_type : (i_op == S_TYPE) ? S_type : I_type
```

9.6.1.20 o_jump_ID

```
assign o_jump_ID = (i_op == J_TYPE_JAL | i_op == I_TYPE_JALR) ? 1'b1 : 1'b0
```

9.6.1.21 o_mem_write_ID

```
assign o_mem_write_ID = (i_op == S_TYPE) ? 1'b1 : 1'b0
```

9.6.1.22 o_reg_write_ID

```
assign o_reg_write_ID
```

Initial value:

```
= (i_op == U_TYPE_LUI | i_op == U_TYPE_AUIPC | i_op == J_TYPE_JAL |  
   i_op == I_TYPE_JALR | i_op == I_TYPE_LOAD | i_op == I_TYPE_ARITH |  
   i_op == R_TYPE) ? 1'b1 : 1'b0
```

9.6.1.23 o_result_src_ID

```
assign o_result_src_ID
```

Initial value:

```
=  
(i_op == R_TYPE) ? 2'b00 :  
(i_op == I_TYPE_LOAD) ? 2'b10 :  
(i_op == I_TYPE_ARITH) ? 2'b00 :  
(i_op == I_TYPE_JALR) ? 2'b01 :  
(i_op == J_TYPE_JAL) ? 2'b01 :  
(i_op == U_TYPE_LUI) ? 2'b00 :  
(i_op == U_TYPE_AUIPC) ? 2'b11 :  
2'bxx
```

9.6.1.24 OP_ADD

```
localparam logic [2:0] OP_ADD = 3'b100
```

9.6.1.25 OP_ADD_SUB

```
localparam logic [2:0] OP_ADD_SUB = 3'b010
```

9.6.1.26 OP_ARITH

```
localparam logic [2:0] OP_ARITH = 3'b001
```

9.6.1.27 OP_BRANCH

```
localparam logic [2:0] OP_BRANCH = 3'b011
```

9.6.1.28 OP_LUI

```
localparam logic [2:0] OP_LUI = 3'b000
```

9.6.1.29 R_TYPE

```
localparam logic [4:0] R_TYPE = 5'b01100
```

9.6.1.30 S_TYPE

```
localparam logic [4:0] S_TYPE = 5'b01000
```

9.6.1.31 S_type

```
localparam logic [2:0] S_type = 3'b001
```

9.6.1.32 U_type

```
localparam logic [2:0] U_type = 3'b100
```

9.6.1.33 U_TYPE_AUIPC

```
localparam logic [4:0] U_TYPE_AUIPC = 5'b00101
```

9.6.1.34 U_TYPE_LUI

```
localparam logic [4:0] U_TYPE_LUI = 5'b01101
```

9.7 tb/s_interface.sv File Reference

Control-unit stimulus/response interface shared by the DUT and the UVM environment.

Functions

- interface [s_interface](#) (input logic clk, input logic reset)

9.7.1 Detailed Description

Control-unit stimulus/response interface shared by the DUT and the UVM environment.

Declares the instruction-field/EX-stage-feedback signals consumed by the control unit and the control signals it produces, and exposes the [driver](#), [monitor](#), and `dut` modports used by the UVM driver, the UVM monitor, and the [control_unit_wrapper](#) DUT respectively.

9.8 tb/tb_top.sv File Reference

Top-level testbench that instantiates the DUT, interface, and UVM test.

Functions

- `s_interface vif (.clk(clk),.reset(rst))`
- `control_unit_wrapper U_CONTROL_UNIT_WRAPPER (.vif(vif))`
- `run_test ("test")`

Variables

- bit `rst`
- `forever`

9.8.1 Detailed Description

Top-level testbench that instantiates the DUT, interface, and UVM test.

Connects `s_interface`, the `control_unit_wrapper` DUT, and the UVM test entry point in a single simulation environment.

9.8.2 Function Documentation

9.8.2.1 run_test()

```
run_test (
    "test" )
```

9.8.2.2 U_CONTROL_UNIT_WRAPPER()

```
control_unit_wrapper U_CONTROL_UNIT_WRAPPER (
    . vifvif)
```

9.8.2.3 vif()

```
s_interface vif (
    . clkclk,
    . resetrst)
```

9.8.3 Variable Documentation

9.8.3.1 forever

```
forever
```

Initial value:

```
{
    rst = 0
```

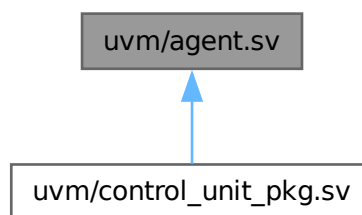
9.8.3.2 rst

```
bit rst
```

9.9 uvm/agent.sv File Reference

UVM agent that bundles the sequencer, driver, and monitor.

This graph shows which files directly or indirectly include this file:



Classes

- class [agent](#)
UVM agent that connects the sequencer, driver, and monitor.

9.9.1 Detailed Description

UVM agent that bundles the sequencer, driver, and monitor.

Implements the standard UVM agent structure for the control-unit verification environment, creating active and passive components as needed.

9.10 agent.sv

[Go to the documentation of this file.](#)

```

00001
00018 class agent extends uvm_agent;
00019     `uvm_component_utils (agent)
00020
00021     driver m_driver;
00022
00023     monitor m_monitor;
00024
00025     sequencer m_sequencer;
00026
00027     function new (string name = "agent", uvm_component parent = null);
00028         super.new (name, parent);
00029     endfunction
  
```

```

00030
00032     virtual function void build_phase (uvm_phase phase);
00033         super.build_phase (phase);
00034
00035         if (get_is_active()) begin
00036             m_sequencer = sequencer::type_id::create ("sequencer", this);
00037             m_driver = driver::type_id::create ("driver", this);
00038         end
00039
00040         m_monitor = monitor::type_id::create ("monitor", this);
00041
00042     endfunction
00043
00044     virtual function void connect_phase (uvm_phase phase);
00045         super.connect_phase (phase);
00046         if (get_is_active()) begin
00047             m_driver.seq_item_port.connect (m_sequencer.seq_item_export);
00048         end
00049     endfunction
00050
00051
00052 endclass

```

9.11 uvm/control_unit_pkg.sv File Reference

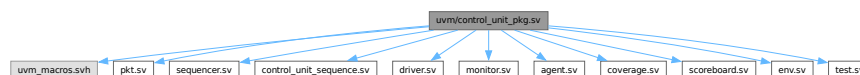
Package that includes the UVM classes used by the control-unit verification environment.

```

#include "uvm_macros.svh"
#include "pkt.sv"
#include "sequencer.sv"
#include "control_unit_sequence.sv"
#include "driver.sv"
#include "monitor.sv"
#include "agent.sv"
#include "coverage.sv"
#include "scoreboard.sv"
#include "env.sv"
#include "test.sv"

```

Include dependency graph for control_unit_pkg.sv:



9.11.1 Detailed Description

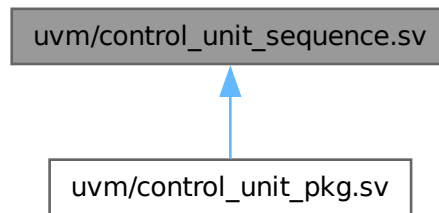
Package that includes the UVM classes used by the control-unit verification environment.

Centralizes the inclusion of all verification components required to build the control-unit UVM testbench.

9.12 uvm/control_unit_sequence.sv File Reference

UVM sequence that drives control-unit transactions until coverage reaches the target.

This graph shows which files directly or indirectly include this file:



Classes

- class `control_unit_sequence`

UVM sequence that drives control-unit transactions until coverage reaches the target.

9.12.1 Detailed Description

UVM sequence that drives control-unit transactions until coverage reaches the target.

Repeatedly sends randomized transactions while waiting for the coverage collector to report completion of the sampling process.

9.13 control_unit_sequence.sv

[Go to the documentation of this file.](#)

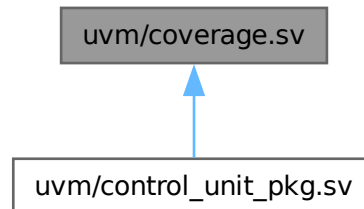
```

00001
00017 class control_unit_sequence extends uvm_sequence #(pkt);
00018     `uvm_object_utils(control_unit_sequence)
00019
00020     real current_coverage = 0;
00021
00022     int num_packets = 0;
00023
00024     function new (string name = "control_unit_sequence");
00025         super.new(name);
00026     endfunction
00027
00029     virtual task body();
00030         pkt packet;
00031         `uvm_create(packet)
00032         while (current_coverage < 100.0) begin
00033             `uvm_rand_send(packet)
00034             num_packets++;
00035             void'(uvm_config_db#(real)::get(null, "*", "cov_status", current_coverage));
00036             `uvm_info("SEQ", $sformatf("Status: %0.2f%%", current_coverage), UVM_LOW)
00037         end
00038         `uvm_info("SEQ", $sformatf("Total packets sent: %0d", num_packets), UVM_LOW)
00039     endtask
00040
00041 endclass
  
```

9.14 uvm/coverage.sv File Reference

Coverage collector for control-unit transactions.

This graph shows which files directly or indirectly include this file:



Classes

- class [coverage](#)
UVM coverage subscriber for control-unit transaction sampling.

9.14.1 Detailed Description

Coverage collector for control-unit transactions.

Samples observed control-unit transactions and updates the coverage model used to determine whether the verification goal has been reached.

9.15 coverage.sv

[Go to the documentation of this file.](#)

```

00001
00019 class coverage extends uvm_subscriber #(pkt);
00020     `uvm_component_utils(coverage)
00021
00022     pkt tr;
00023
00025     covergroup cg_control_unit;
00026         option.per_instance = 1;
00027
00029         cp_op: coverpoint tr.i_op {
00030             bins op_load    = {5'b00000};
00031             bins op_imm     = {5'b00100};
00032             bins op_auipc   = {5'b00101};
00033             bins op_store   = {5'b01000};
00034             bins op_reg     = {5'b01100};
00035             bins op_lui     = {5'b01101};
00036             bins op_branch  = {5'b11000};
00037             bins op_jalr    = {5'b11001};
00038             bins op_jal     = {5'b11011};
00039         }
00040
00042         cp_funcnt3: coverpoint tr.i_funcnt_3 {
00043             bins values[8] = {[0:7]};
  
```

```

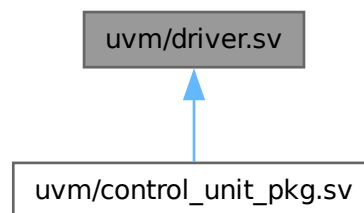
00044     }
00045
00047     cp_funct7_5: coverpoint tr.i_funct_7_5 {
00048         bins values[2] = {[0:1]};
00049     }
00050
00054     cp_branch_ex: coverpoint tr.i_branch_EX {
00055         bins values[2] = {[0:1]};
00056     }
00057
00059     cp_jump_ex: coverpoint tr.i_jump_EX {
00060         bins values[2] = {[0:1]};
00061     }
00062
00064     cp_zero: coverpoint tr.i_zero {
00065         bins values[2] = {[0:1]};
00066     }
00067
00069     cp_cross_pc_src: cross cp_branch_ex, cp_jump_ex, cp_zero;
00070
00072     cp_cross_decode: cross cp_op, cp_funct3, cp_funct7_5 {
00073         // funct3 == 3'b010/3'b011 are reserved (not real branch conditions) and
00074         // excluded by pkt.sv's branch_funct3_valid constraint, so these bins can
00075         // never be hit.
00076         ignore_bins reserved_branch_funct3 =
00077             binsof(cp_op.op_branch) && binsof(cp_funct3) intersect {2, 3};
00078     }
00079
00080 endgroup
00081
00082 function new(string name = "coverage", uvm_component parent);
00083     super.new(name, parent);
00084     cg_control_unit = new();
00085     cg_control_unit.set_inst_name("control_unit_cov");
00086 endfunction
00087
00089 virtual function void write(pkt t);
00090     this.tr = t;
00091     cg_control_unit.sample();
00092     uvm_config_db#(real)::set(null, "*", "cov_status", cg_control_unit.get_inst_coverage());
00093 endfunction
00094
00095 endclass

```

9.16 uvm/driver.sv File Reference

UVM driver that translates control-unit transactions into interface signal activity.

This graph shows which files directly or indirectly include this file:



Classes

- class [driver](#)

UVM driver for control-unit transactions.

9.16.1 Detailed Description

UVM driver that translates control-unit transactions into interface signal activity.

Receives sequence items from the sequencer and drives the instruction fields/EX-stage feedback onto [s_interface](#) for one clock cycle each.

9.17 driver.sv

[Go to the documentation of this file.](#)

```

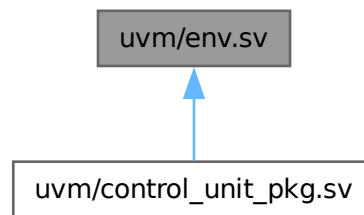
00001
00017 class driver extends uvm_driver #(pkt);
00018     `uvm_component_utils (driver)
00019
00020     virtual s_interface.driver vif;
00021
00022     pkt req_pkt;
00023
00024     function new (string name = "driver", uvm_component parent);
00025         super.new(name, parent);
00026     endfunction
00027
00029     virtual function void build_phase (uvm_phase phase);
00030         super.build_phase(phase);
00031         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00032             `uvm_fatal (get_type_name (), "Didn't get handle to virtual interface if_name")
00033         end
00034     endfunction
00035
00037     virtual task run_phase(uvm_phase phase);
00038         super.run_phase(phase);
00039
00040         wait (vif.reset == 1'b1);
00041
00042         forever begin
00043             seq_item_port.get_next_item(req_pkt);
00044             drive_item(req_pkt);
00045             seq_item_port.item_done();
00046         end
00047     endtask
00048
00050     virtual task drive_item (pkt pkt_item);
00051         @(posedge vif.clk);
00052         vif.op          <= pkt_item.i_op;
00053         vif.funct3       <= pkt_item.i_funct_3;
00054         vif.funct7_5     <= pkt_item.i_funct_7_5;
00055         vif.branch_ex   <= pkt_item.i_branch_EX;
00056         vif.jump_ex     <= pkt_item.i_jump_EX;
00057         vif.zero        <= pkt_item.i_zero;
00058         vif.enable      <= '1;
00059
00060         @(posedge vif.clk);
00061         vif.enable <= '0;
00062     endtask
00063
00064 endclass

```

9.18 uvm/env.sv File Reference

UVM environment that instantiates the agent, scoreboard, and coverage collector.

This graph shows which files directly or indirectly include this file:



Classes

- class `env`

Top-level UVM environment for the control-unit verification bench.

9.18.1 Detailed Description

UVM environment that instantiates the agent, scoreboard, and coverage collector.

Connects the main verification components so that transactions generated by the agent can be checked and tracked by the scoreboard and coverage.

9.19 env.sv

[Go to the documentation of this file.](#)

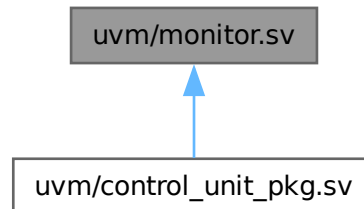
```

00001
00016 class env extends uvm_env;
00017     `uvm_component_utils (env)
00018
00019     agent m_agent;
00020
00021     scoreboard m_scoreboard;
00022
00023     coverage m_coverage;
00024
00025     function new (string name = "env", uvm_component parent = null);
00026         super.new (name, parent);
00027     endfunction
00028
00030     virtual function void build_phase (uvm_phase phase);
00031         super.build_phase (phase);
00032         m_agent = agent::type_id::create ("agent", this);
00033         m_scoreboard = scoreboard::type_id::create ("scoreboard", this);
00034         m_coverage = coverage::type_id::create ("coverage", this);
00035     endfunction
00036
00038     virtual function void connect_phase (uvm_phase phase);
00039         super.connect_phase (phase);
00040         m_agent.m_monitor.mon_analysis_port.connect (m_scoreboard.ap_imp);
00041         m_agent.m_monitor.mon_analysis_port.connect (m_coverage.analysis_export);
00042     endfunction
00043
00044 endclass
  
```

9.20 uvm/monitor.sv File Reference

UVM monitor that samples control-unit transactions and publishes them to subscribers.

This graph shows which files directly or indirectly include this file:



Classes

- class [monitor](#)
UVM monitor for control-unit transaction observation.

9.20.1 Detailed Description

UVM monitor that samples control-unit transactions and publishes them to subscribers.

Observes [s_interface](#) for each transaction and forwards the completed input/output fields to the scoreboard and coverage collector.

9.21 monitor.sv

[Go to the documentation of this file.](#)

```

00001
00018 class monitor extends uvm_monitor;
00019     `uvm_component_utils(monitor)
00020
00021     virtual s_interface.monitor vif;
00022
00023     uvm_analysis_port #(pkt) mon_analysis_port;
00024
00025     function new(string name = "monitor", uvm_component parent = null);
00026         super.new(name, parent);
00027     endfunction
00028
00030     virtual function void build_phase(uvm_phase phase);
00031         super.build_phase(phase);
00032         mon_analysis_port = new("mon_analysis_port", this);
00033
00034         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00035             `uvm_error(get_type_name(), "DUT interface not found")
00036         end
00037     endfunction
00038
00048     virtual task run_phase(uvm_phase phase);
00049         super.run_phase(phase);
  
```

```

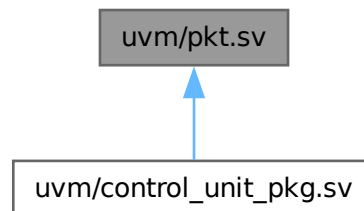
00050
00051     forever begin
00052         pkt mon_pkt;
00053         mon_pkt = pkt::type_id::create("mon_pkt", this);
00054
00055         @(posedge vif.enable);
00056         #1;
00057
00058         mon_pkt.i_op          = vif.op;
00059         mon_pkt.i_funct_3     = vif.funct3;
00060         mon_pkt.i_funct_7_5   = vif.funct7_5;
00061         mon_pkt.i_branch_EX   = vif.branch_ex;
00062         mon_pkt.i_jump_EX     = vif.jump_ex;
00063         mon_pkt.i_zero        = vif.zero;
00064
00065         mon_pkt.o_pc_src_EX   = vif.pc_src_ex;
00066         mon_pkt.o_jump_ID     = vif.jump_id;
00067         mon_pkt.o_branch_ID   = vif.branch_id;
00068         mon_pkt.o_reg_write_ID = vif.reg_write_id;
00069         mon_pkt.o_result_src_ID = vif.result_src_id;
00070         mon_pkt.o_mem_write_ID = vif.mem_write_id;
00071         mon_pkt.o_alu_ctrl_ID = vif.alu_ctrl_id;
00072         mon_pkt.o_alu_src_ID  = vif.alu_src_id;
00073         mon_pkt.o_addr_src_ID = vif.addr_src_id;
00074         mon_pkt.o_imm_src_ID  = vif.imm_src_id;
00075         mon_pkt.o_fence_ID    = vif.fence_id;
00076
00077         `uvm_info(get_type_name(),
00078             $sformatf("Monitored op=%0b f3=%0d f7_5=%0b branchEX=%0b jumpEX=%0b zero=%0b ->
pcSrc=%0b jump=%0b branch=%0b regW=%0b resSrc=%0d memW=%0b aluCtrl=%0d aluSrc=%0b addrSrc=%0b
immSrc=%0d fence=%0b",
mon_pkt.i_op, mon_pkt.i_funct_3, mon_pkt.i_funct_7_5, mon_pkt.i_branch_EX,
mon_pkt.i_jump_EX, mon_pkt.i_zero, mon_pkt.o_pc_src_EX, mon_pkt.o_jump_ID,
mon_pkt.o_branch_ID, mon_pkt.o_reg_write_ID, mon_pkt.o_result_src_ID,
mon_pkt.o_mem_write_ID, mon_pkt.o_alu_ctrl_ID, mon_pkt.o_alu_src_ID,
mon_pkt.o_addr_src_ID, mon_pkt.o_imm_src_ID, mon_pkt.o_fence_ID),
UVM_LOW)
00085
00086         mon_analysis_port.write(mon_pkt);
00087     end
00088 endtask
00089
00090 endclass

```

9.22 uvm/pkt.sv File Reference

Transaction item used to model one control-unit decode in the UVM environment.

This graph shows which files directly or indirectly include this file:



Classes

- class `pkt`

Sequence item that models one control-unit decode (stimulus + expected response fields).

9.22.1 Detailed Description

Transaction item used to model one control-unit decode in the UVM environment.

Encapsulates the instruction fields and EX-stage feedback driven into the control unit, plus the control signals sampled back from it.

9.23 pkt.sv

[Go to the documentation of this file.](#)

```

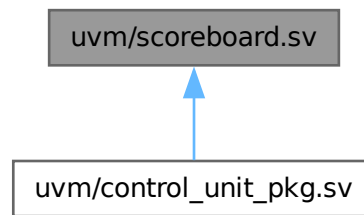
00001
00017 class pkt extends uvm_sequence_item;
00018
00019     rand logic [4:0] i_op;
00020     rand logic [2:0] i_funct_3;
00021     rand logic      i_funct_7_5;
00022     rand logic      i_branch_EX;
00023     rand logic      i_jump_EX;
00024     rand logic      i_zero;
00025
00026     logic          o_pc_src_EX;
00027     logic          o_jump_ID;
00028     logic          o_branch_ID;
00029     logic          o_reg_write_ID;
00030     logic [1:0]    o_result_src_ID;
00031     logic          o_mem_write_ID;
00032     logic [4:0]    o_alu_ctrl_ID;
00033     logic          o_alu_src_ID;
00034     logic          o_addr_src_ID;
00035     logic [2:0]    o_imm_src_ID;
00036     logic          o_fence_ID;
00037
00038     `uvm_object_utils_begin(pkt)
00039         `uvm_field_int (i_op, UVM_DEFAULT)
00040         `uvm_field_int (i_funct_3, UVM_DEFAULT)
00041         `uvm_field_int (i_funct_7_5, UVM_DEFAULT)
00042         `uvm_field_int (i_branch_EX, UVM_DEFAULT)
00043         `uvm_field_int (i_jump_EX, UVM_DEFAULT)
00044         `uvm_field_int (i_zero, UVM_DEFAULT)
00045         `uvm_field_int (o_pc_src_EX, UVM_DEFAULT)
00046         `uvm_field_int (o_jump_ID, UVM_DEFAULT)
00047         `uvm_field_int (o_branch_ID, UVM_DEFAULT)
00048         `uvm_field_int (o_reg_write_ID, UVM_DEFAULT)
00049         `uvm_field_int (o_result_src_ID, UVM_DEFAULT)
00050         `uvm_field_int (o_mem_write_ID, UVM_DEFAULT)
00051         `uvm_field_int (o_alu_ctrl_ID, UVM_DEFAULT)
00052         `uvm_field_int (o_alu_src_ID, UVM_DEFAULT)
00053         `uvm_field_int (o_addr_src_ID, UVM_DEFAULT)
00054         `uvm_field_int (o_imm_src_ID, UVM_DEFAULT)
00055         `uvm_field_int (o_fence_ID, UVM_DEFAULT)
00056     `uvm_object_utils_end
00057
00062     constraint op_range {
00063         i_op inside {5'b00000, 5'b00100, 5'b00101, 5'b01000,
00064                     5'b01100, 5'b01101, 5'b11000, 5'b11001, 5'b11011};
00065     }
00066
00069     constraint branch_funct3_valid {
00070         (i_op == 5'b11000) -> i_funct_3 inside {3'b000, 3'b001, 3'b100, 3'b101, 3'b110, 3'b111};
00071     }
00072
00073     function new(string name = "pkt");
00074         super.new(name);
00075     endfunction
00076
00077 endclass

```

9.24 uvm/scoreboard.sv File Reference

Scoreboard that checks control-unit transactions against the golden reference model.

This graph shows which files directly or indirectly include this file:



Classes

- class [scoreboard](#)
UVM scoreboard for control-unit functional checking.

Functions

- void [control_unit_golden](#) (input int unsigned op, input int unsigned funct3, input byte unsigned funct7_5, input byte unsigned branch_ex, input byte unsigned jump_ex, input byte unsigned zero, output byte unsigned exp_jump_id, output byte unsigned exp_branch_id, output byte unsigned exp_reg_write_id, output int unsigned exp_result_src_id, output byte unsigned exp_mem_write_id, output int unsigned exp_alu_ctrl_id, output byte unsigned exp_alu_src_id, output byte unsigned exp_addr_src_id, output int unsigned exp_imm_src_id, output byte unsigned exp_fence_id, output byte unsigned exp_pc_src_ex)
DPI-C import of the golden reference model (see [reference_model.c](#)).

9.24.1 Detailed Description

Scoreboard that checks control-unit transactions against the golden reference model.

Receives monitored transactions and validates them against the DPI-C reference model implemented in the C source file.

9.24.2 Function Documentation

9.24.2.1 control_unit_golden()

```

void control_unit_golden (
    input int unsigned op,
    input int unsigned funct3,
    input byte unsigned funct7_5,
    input byte unsigned branch_ex,
    input byte unsigned jump_ex,
    input byte unsigned zero,
    output byte unsigned exp_jump_id,

```

```

output byte unsigned exp_branch_id,
output byte unsigned exp_reg_write_id,
output int unsigned exp_result_src_id,
output byte unsigned exp_mem_write_id,
output int unsigned exp_alu_ctrl_id,
output byte unsigned exp_alu_src_id,
output byte unsigned exp_addr_src_id,
output int unsigned exp_imm_src_id,
output byte unsigned exp_fence_id,
output byte unsigned exp_pc_src_ex)

```

DPI-C import of the golden reference model (see [reference_model.c](#)).

Parameters

<i>op</i>	instr[6:2].
<i>funct3</i>	instr[14:12].
<i>funct7_5</i>	instr[30].
<i>branch_ex</i>	Registered o_branch_ID at Execute.
<i>jump_ex</i>	Registered o_jump_ID at Execute.
<i>zero</i>	ALU branch-comparison result.
<i>exp_jump_id</i>	Output: expected o_jump_ID.
<i>exp_branch_id</i>	Output: expected o_branch_ID.
<i>exp_reg_write_id</i>	Output: expected o_reg_write_ID.
<i>exp_result_src_id</i>	Output: expected o_result_src_ID.
<i>exp_mem_write_id</i>	Output: expected o_mem_write_ID.
<i>exp_alu_ctrl_id</i>	Output: expected o_alu_ctrl_ID.
<i>exp_alu_src_id</i>	Output: expected o_alu_src_ID.
<i>exp_addr_src_id</i>	Output: expected o_addr_src_ID.
<i>exp_imm_src_id</i>	Output: expected o_imm_src_ID.
<i>exp_fence_id</i>	Output: expected o_fence_ID.
<i>exp_pc_src_ex</i>	Output: expected o_pc_src_EX.

9.25 scoreboard.sv

[Go to the documentation of this file.](#)

00001

```

00030 import "DPI-C" function void control_unit_golden(
00031     input  int unsigned op,
00032     input  int unsigned funct3,
00033     input  byte unsigned funct7_5,
00034     input  byte unsigned branch_ex,
00035     input  byte unsigned jump_ex,
00036     input  byte unsigned zero,
00037     output byte unsigned exp_jump_id,
00038     output byte unsigned exp_branch_id,
00039     output byte unsigned exp_reg_write_id,
00040     output int unsigned exp_result_src_id,
00041     output byte unsigned exp_mem_write_id,
00042     output int unsigned exp_alu_ctrl_id,
00043     output byte unsigned exp_alu_src_id,
00044     output byte unsigned exp_addr_src_id,
00045     output int unsigned exp_imm_src_id,
00046     output byte unsigned exp_fence_id,
00047     output byte unsigned exp_pc_src_ex
00048 );
00049
00057 class scoreboard extends uvm_scoreboard;
00058     `uvm_component_utils(scoreboard)
00059
00060     uvm_analysis_imp #(pkt, scoreboard) ap_imp;
00061
00062     int num_errors = 0;
00063
00064     function new(string name = "scoreboard", uvm_component parent = null);
00065         super.new(name, parent);
00066     endfunction
00067
00069     virtual function void build_phase(uvm_phase phase);
00070         super.build_phase(phase);
00071         ap_imp = new("ap_imp", this);
00072     endfunction
00073
00075     virtual function void write(pkt data);
00076         byte unsigned jump_raw, branch_raw, reg_write_raw, mem_write_raw;
00077         byte unsigned alu_src_raw, addr_src_raw, fence_raw, pc_src_raw;
00078         logic [1:0] exp_result_src_id;
00079         logic [4:0] exp_alu_ctrl_id;
00080         logic [2:0] exp_imm_src_id;
00081         logic exp_jump_id, exp_branch_id, exp_reg_write_id, exp_mem_write_id;
00082         logic exp_alu_src_id, exp_addr_src_id, exp_fence_id, exp_pc_src_ex;
00083
00084         control_unit_golden(data.i_op, data.i_funct3, data.i_funct7_5, data.i_branch_EX,
00085             data.i_jump_EX, data.i_zero,
00086             jump_raw, branch_raw, reg_write_raw, exp_result_src_id, mem_write_raw, exp_alu_ctrl_id,
00087             alu_src_raw, addr_src_raw, exp_imm_src_id, fence_raw, pc_src_raw);
00088
00089         exp_jump_id = jump_raw[0];
00090         exp_branch_id = branch_raw[0];
00091         exp_reg_write_id = reg_write_raw[0];
00092         exp_mem_write_id = mem_write_raw[0];
00093         exp_alu_src_id = alu_src_raw[0];
00094         exp_addr_src_id = addr_src_raw[0];
00095         exp_fence_id = fence_raw[0];
00096         exp_pc_src_ex = pc_src_raw[0];
00097
00098         if (data.o_jump_ID == exp_jump_id && data.o_branch_ID == exp_branch_id &&
00099             data.o_reg_write_ID == exp_reg_write_id && data.o_result_src_ID == exp_result_src_id &&
00100             data.o_mem_write_ID == exp_mem_write_id && data.o_alu_ctrl_ID == exp_alu_ctrl_id &&
00101             data.o_alu_src_ID == exp_alu_src_id &&
00102             data.o_addr_src_ID == exp_addr_src_id && data.o_imm_src_ID == exp_imm_src_id &&
00103             data.o_fence_ID == exp_fence_id && data.o_pc_src_EX == exp_pc_src_ex) begin
00104             `uvm_info("SCOREBOARD",
00105                 $sformatf("PASS: op=%0b f3=%0d f7_5=%0b -> jump=%0b branch=%0b regW=%0b resSrc=%0d memW=%0b
00106 aluCtrl=%0d aluSrc=%0b addrSrc=%0b immSrc=%0d fence=%0b pcSrc=%0b",
00107                     data.i_op, data.i_funct3, data.i_funct7_5, data.o_jump_ID, data.o_branch_ID,
00108                     data.o_reg_write_ID, data.o_result_src_ID, data.o_mem_write_ID, data.o_alu_ctrl_ID,
00109                     data.o_alu_src_ID, data.o_addr_src_ID, data.o_imm_src_ID, data.o_fence_ID,
00110                     data.o_pc_src_EX),
00111                     UVM_LOW)
00112             end
00113         else begin
00114             `uvm_error("SCOREBOARD",
00115                 $sformatf("FAIL: op=%0b f3=%0d f7_5=%0b branchEX=%0b jumpEX=%0b zero=%0b EXPECTED jump=%0b
00116 branch=%0b regW=%0b resSrc=%0d memW=%0b aluCtrl=%0d aluSrc=%0b addrSrc=%0b immSrc=%0d fence=%0b
00117 pcSrc=%0b GOT jump=%0b branch=%0b regW=%0b resSrc=%0d memW=%0b aluCtrl=%0d aluSrc=%0b addrSrc=%0b
00118 immSrc=%0d fence=%0b pcSrc=%0b",
00119                     data.i_op, data.i_funct3, data.i_funct7_5, data.i_branch_EX, data.i_jump_EX, data.i_zero,
00120                     exp_jump_id, exp_branch_id, exp_reg_write_id, exp_result_src_id, exp_mem_write_id,
00121                     exp_alu_ctrl_id, exp_alu_src_id, exp_addr_src_id, exp_imm_src_id, exp_fence_id,
00122                     exp_pc_src_ex,
00123                     data.o_jump_ID, data.o_branch_ID, data.o_reg_write_ID, data.o_result_src_ID,
00124                     data.o_mem_write_ID, data.o_alu_ctrl_ID, data.o_alu_src_ID, data.o_addr_src_ID,
00125                     data.o_imm_src_ID, data.o_fence_ID, data.o_pc_src_EX))

```

```

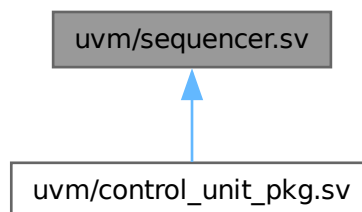
00120         this.num_errors++;
00121     end
00122 endfunction
00123
00124 virtual function void check_phase(uvm_phase phase);
00125     super.check_phase(phase);
00126     if (this.num_errors > 0) begin
00127         `uvm_fatal("FINAL_RESULT",
00128             $sformatf("TEST FAILED: Scoreboard found %0d errors.", num_errors))
00129     end
00130     else begin
00131         `uvm_info("FINAL_RESULT", "TEST PASS: All transactions were correct.", UVM_NONE)
00132     end
00133 endfunction
00134
00135 virtual task run_phase(uvm_phase phase);
00136     super.run_phase(phase);
00137 endtask
00138
00139 endclass

```

9.26 uvm/sequencer.sv File Reference

UVM sequencer that provides transactions to the driver.

This graph shows which files directly or indirectly include this file:



Classes

- class `sequencer`
UVM sequencer for control-unit transactions.

9.26.1 Detailed Description

UVM sequencer that provides transactions to the driver.

Acts as the transaction source for the UVM driver in the control-unit verification.

9.27 sequencer.sv

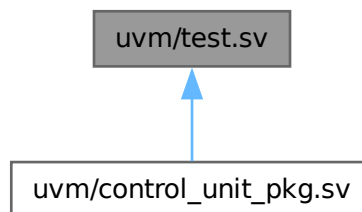
[Go to the documentation of this file.](#)

```
00001
00014 class sequencer extends uvm_sequencer #(pkt);
00015
00016     `uvm_component_utils (sequencer)
00017
00018     function new (string name="sequencer", uvm_component parent);
00019         super.new (name, parent);
00020     endfunction
00021
00022 endclass
```

9.28 uvm/test.sv File Reference

Top-level UVM test that builds the environment and starts the main sequence.

This graph shows which files directly or indirectly include this file:



Classes

- class [test](#)
Top-level UVM test for the control-unit verification bench.

9.28.1 Detailed Description

Top-level UVM test that builds the environment and starts the main sequence.

Instantiates the verification environment and launches the control-unit sequence that drives the DUT under test.

9.29 test.sv

[Go to the documentation of this file.](#)

```
00001
00016 class test extends uvm_test;
00017     `uvm_component_utils (test)
00018
00019     env m_env;
00020
00021     control_unit_sequence m_sequence;
00022
00023     function new (string name = "test", uvm_component parent = null);
00024         super.new (name, parent);
00025     endfunction
00026
00028     virtual function void build_phase (uvm_phase phase);
00029         super.build_phase (phase);
00030         m_env = env::type_id::create ("env", this);
00031     endfunction
00032
00034     virtual function void end_of_elaboration_phase (uvm_phase phase);
00035         uvm_root::get().print_topology();
00036     endfunction
00037
00039     virtual task run_phase(uvm_phase phase);
00040         super.run_phase(phase);
00041
00042         m_sequence = control_unit_sequence::type_id::create("sequence");
00043
00044         phase.raise_objection(this);
00045         m_sequence.start(m_env.m_agent.m_sequencer);
00046         phase.drop_objection(this);
00047     endtask
00048 endclass
```

Index

- ADD
 - alu_decoder.v, [50](#)
- agent, [17](#)
 - build_phase, [18](#)
 - connect_phase, [18](#)
 - m_monitor, [19](#)
 - m_sequencer, [19](#)
 - new, [18](#)
 - uvm_component_utils, [18](#)
- ALU_ADD
 - reference_model.c, [44](#)
- ALU_AND
 - reference_model.c, [44](#)
- ALU_BEQ
 - reference_model.c, [45](#)
- ALU_BGE
 - reference_model.c, [45](#)
- ALU_BGEU
 - reference_model.c, [45](#)
- ALU_BLT
 - reference_model.c, [45](#)
- ALU_BLTU
 - reference_model.c, [45](#)
- ALU_BNE
 - reference_model.c, [45](#)
- alu_decoder.v
 - ADD, [50](#)
 - AND, [50](#)
 - AUIPC, [50](#)
 - BEQ, [50](#)
 - BGE, [50](#)
 - BGEU, [50](#)
 - BLT, [50](#)
 - BLTU, [50](#)
 - BNE, [51](#)
 - EBREAK, [51](#)
 - ECALL, [51](#)
 - FENCE, [51](#)
 - i_alu_op, [51](#)
 - i_funct_3, [51](#)
 - i_funct_7_5, [51](#)
 - LUI, [51](#)
 - o_alu_ctrl_ID, [51](#)
 - OP_ADD, [52](#)
 - OP_ADD_SUB, [52](#)
 - OP_ARITH, [52](#)
 - OP_BRANCH, [52](#)
 - OP_LUI, [52](#)
 - OR, [52](#)
 - SLL, [52](#)
 - SLT, [53](#)
 - SLTU, [53](#)
 - SRA, [53](#)
 - SRL, [53](#)
 - SUB, [53](#)
 - XOR, [53](#)
- ALU_LUI
 - reference_model.c, [45](#)
- alu_op
 - control_unit.v, [54](#)
- ALU_OR
 - reference_model.c, [45](#)
- ALU_SLL
 - reference_model.c, [45](#)
- ALU_SLT
 - reference_model.c, [45](#)
- ALU_SLTU
 - reference_model.c, [46](#)
- ALU_SRA
 - reference_model.c, [46](#)
- ALU_SRL
 - reference_model.c, [46](#)
- ALU_SUB
 - reference_model.c, [46](#)
- ALU_XOR
 - reference_model.c, [46](#)
- AND
 - alu_decoder.v, [50](#)
- ap_imp
 - scoreboard, [38](#)
- AUIPC
 - alu_decoder.v, [50](#)
- B_type
 - op_decoder.v, [58](#)
- B_TYPE_BRANCH
 - op_decoder.v, [58](#)
- BEQ
 - alu_decoder.v, [50](#)
- BGE
 - alu_decoder.v, [50](#)
- BGEU
 - alu_decoder.v, [50](#)
- BLT
 - alu_decoder.v, [50](#)
- BLTU
 - alu_decoder.v, [50](#)
- BNE
 - alu_decoder.v, [51](#)

- body
 - control_unit_sequence, 20
- branch_funct3_valid
 - pkt, 32
- build_phase
 - agent, 18
 - driver, 25
 - env, 27
 - monitor, 29
 - scoreboard, 37
 - test, 41
- cg_control_unit
 - coverage, 22
- check_phase
 - scoreboard, 37
- connect_phase
 - agent, 18
 - env, 27
- control_unit.v
 - alu_op, 54
 - i_branch_EX, 54
 - i_funct_3, 55
 - i_funct_7_5, 55
 - i_jump_EX, 55
 - i_op, 55
 - i_zero, 55
 - o_addr_src_ID, 55
 - o_alu_ctrl_ID, 55
 - o_alu_src_ID, 55
 - o_branch_ID, 55
 - o_fence_ID, 56
 - o_imm_src_ID, 56
 - o_jump_ID, 56
 - o_mem_write_ID, 56
 - o_pc_src_EX, 56
 - o_reg_write_ID, 56
 - o_result_src_ID, 56
 - U_ALU_DECODER, 54
 - U_OP_DECODER, 54
- control_unit_golden
 - reference_model.c, 48
 - scoreboard.sv, 73
- control_unit_sequence, 19
 - body, 20
 - new, 20
 - num_packets, 21
 - uvm_object_utils, 20
- coverage, 21
 - cg_control_unit, 22
 - new, 22
 - uvm_component_utils, 22
 - write, 23
- drive_item
 - driver, 25
- driver, 23
 - build_phase, 25
 - drive_item, 25
 - new, 25
 - req_pkt, 25
 - run_phase, 25
 - uvm_component_utils, 25
- EBREAK
 - alu_decoder.v, 51
- ECALL
 - alu_decoder.v, 51
- end_of_elaboration_phase
 - test, 41
- env, 26
 - build_phase, 27
 - connect_phase, 27
 - m_coverage, 27
 - m_scoreboard, 27
 - new, 27
 - uvm_component_utils, 27
- FENCE
 - alu_decoder.v, 51
- filter Directory Reference, 15
- filter/README.md, 43
- forever
 - tb_top.sv, 62
- i_alu_op
 - alu_decoder.v, 51
- i_branch_EX
 - control_unit.v, 54
 - pkt, 34
- i_funct_3
 - alu_decoder.v, 51
 - control_unit.v, 55
 - op_decoder.v, 58
 - pkt, 34
- i_funct_7_5
 - alu_decoder.v, 51
 - control_unit.v, 55
 - op_decoder.v, 58
 - pkt, 34
- i_jump_EX
 - control_unit.v, 55
 - pkt, 34
- i_op
 - control_unit.v, 55
 - op_decoder.v, 58
 - pkt, 34
- I_type
 - op_decoder.v, 58
- I_TYPE_ARITH
 - op_decoder.v, 58
- I_TYPE_FENCE
 - op_decoder.v, 58
- I_TYPE_JALR
 - op_decoder.v, 58
- I_TYPE_LOAD
 - op_decoder.v, 58
- I_TYPE_SYSTEM

- op_decoder.v, 58
- i_zero
 - control_unit.v, 55
 - pkt, 34
- IMM_B
 - reference_model.c, 46
- IMM_I
 - reference_model.c, 46
- IMM_J
 - reference_model.c, 46
- IMM_S
 - reference_model.c, 46
- IMM_U
 - reference_model.c, 46
- Interfaces, 12
 - s_interface, 12
- J_type
 - op_decoder.v, 59
- J_TYPE_JAL
 - op_decoder.v, 59
- LUI
 - alu_decoder.v, 51
- m_coverage
 - env, 27
- m_monitor
 - agent, 19
- m_scoreboard
 - env, 27
- m_sequence
 - test, 42
- m_sequencer
 - agent, 19
- Modules, 11
- mon_analysis_port
 - monitor, 30
- monitor, 28
 - build_phase, 29
 - mon_analysis_port, 30
 - new, 29
 - run_phase, 29
 - uvm_component_utils, 29
- new
 - agent, 18
 - control_unit_sequence, 20
 - coverage, 22
 - driver, 25
 - env, 27
 - monitor, 29
 - pkt, 32
 - scoreboard, 38
 - test, 42
- num_errors
 - scoreboard, 38
- num_packets
 - control_unit_sequence, 21
- o_addr_src_ID
 - control_unit.v, 55
 - op_decoder.v, 59
 - pkt, 34
- o_alu_ctrl_ID
 - alu_decoder.v, 51
 - control_unit.v, 55
 - pkt, 34
- o_alu_op
 - op_decoder.v, 59
- o_alu_src_ID
 - control_unit.v, 55
 - op_decoder.v, 59
 - pkt, 35
- o_branch_ID
 - control_unit.v, 55
 - op_decoder.v, 59
 - pkt, 35
- o_fence_ID
 - control_unit.v, 56
 - op_decoder.v, 59
 - pkt, 35
- o_imm_src_ID
 - control_unit.v, 56
 - op_decoder.v, 59
 - pkt, 35
- o_jump_ID
 - control_unit.v, 56
 - op_decoder.v, 59
 - pkt, 35
- o_mem_write_ID
 - control_unit.v, 56
 - op_decoder.v, 60
 - pkt, 35
- o_pc_src_EX
 - control_unit.v, 56
 - pkt, 35
- o_reg_write_ID
 - control_unit.v, 56
 - op_decoder.v, 60
 - pkt, 35
- o_result_src_ID
 - control_unit.v, 56
 - op_decoder.v, 60
 - pkt, 36
- OP_ADD
 - alu_decoder.v, 52
 - op_decoder.v, 60
- OP_ADD_SUB
 - alu_decoder.v, 52
 - op_decoder.v, 60
- OP_ARITH
 - alu_decoder.v, 52
 - op_decoder.v, 60
- OP_AUIPC
 - reference_model.c, 47
- OP_BRANCH
 - alu_decoder.v, 52

- op_decoder.v, 60
 - reference_model.c, 47
- op_decoder.v
 - B_type, 58
 - B_TYPE_BRANCH, 58
 - i_funct_3, 58
 - i_funct_7_5, 58
 - i_op, 58
 - I_type, 58
 - I_TYPE_ARITH, 58
 - I_TYPE_FENCE, 58
 - I_TYPE_JALR, 58
 - I_TYPE_LOAD, 58
 - I_TYPE_SYSTEM, 58
 - J_type, 59
 - J_TYPE_JAL, 59
 - o_addr_src_ID, 59
 - o_alu_op, 59
 - o_alu_src_ID, 59
 - o_branch_ID, 59
 - o_fence_ID, 59
 - o_imm_src_ID, 59
 - o_jump_ID, 59
 - o_mem_write_ID, 60
 - o_reg_write_ID, 60
 - o_result_src_ID, 60
 - OP_ADD, 60
 - OP_ADD_SUB, 60
 - OP_ARITH, 60
 - OP_BRANCH, 60
 - OP_LUI, 60
 - R_TYPE, 61
 - S_TYPE, 61
 - S_type, 61
 - U_type, 61
 - U_TYPE_AUIPC, 61
 - U_TYPE_LUI, 61
- OP_IMM
 - reference_model.c, 47
- OP_JAL
 - reference_model.c, 47
- OP_JALR
 - reference_model.c, 47
- OP_LOAD
 - reference_model.c, 47
- OP_LUI
 - alu_decoder.v, 52
 - op_decoder.v, 60
 - reference_model.c, 47
- op_range
 - pkt, 32
- OP_REG
 - reference_model.c, 47
- OP_STORE
 - reference_model.c, 47
- OR
 - alu_decoder.v, 52
- parent
 - sequencer, 40
- pkt, 30
 - branch_funct3_valid, 32
 - i_branch_EX, 34
 - i_funct_3, 34
 - i_funct_7_5, 34
 - i_jump_EX, 34
 - i_op, 34
 - i_zero, 34
 - new, 32
 - o_addr_src_ID, 34
 - o_alu_ctrl_ID, 34
 - o_alu_src_ID, 35
 - o_branch_ID, 35
 - o_fence_ID, 35
 - o_imm_src_ID, 35
 - o_jump_ID, 35
 - o_mem_write_ID, 35
 - o_pc_src_EX, 35
 - o_reg_write_ID, 35
 - o_result_src_ID, 36
 - op_range, 32
 - uvm_field_int, 32, 33
 - uvm_object_utils_begin, 33
- R_TYPE
 - op_decoder.v, 61
- reference_model Directory Reference, 15
- reference_model.c
 - ALU_ADD, 44
 - ALU_AND, 44
 - ALU_BEQ, 45
 - ALU_BGE, 45
 - ALU_BGEU, 45
 - ALU_BLT, 45
 - ALU_BLTU, 45
 - ALU_BNE, 45
 - ALU_LUI, 45
 - ALU_OR, 45
 - ALU_SLL, 45
 - ALU_SLT, 45
 - ALU_SLTU, 46
 - ALU_SRA, 46
 - ALU_SRL, 46
 - ALU_SUB, 46
 - ALU_XOR, 46
 - control_unit_golden, 48
 - IMM_B, 46
 - IMM_I, 46
 - IMM_J, 46
 - IMM_S, 46
 - IMM_U, 46
 - OP_AUIPC, 47
 - OP_BRANCH, 47
 - OP_IMM, 47
 - OP_JAL, 47
 - OP_JALR, 47
 - OP_LOAD, 47
 - OP_LUI, 47

- OP_REG, [47](#)
- OP_STORE, [47](#)
- RESULT_ALU, [47](#)
- RESULT_MEM, [48](#)
- RESULT_PC4, [48](#)
- RESULT_PCIMM, [48](#)
- reference_model/reference_model.c, [43](#)
- req_pkt
 - driver, [25](#)
- RESULT_ALU
 - reference_model.c, [47](#)
- RESULT_MEM
 - reference_model.c, [48](#)
- RESULT_PC4
 - reference_model.c, [48](#)
- RESULT_PCIMM
 - reference_model.c, [48](#)
- rst
 - tb_top.sv, [62](#)
- RTL design, [11](#)
- rtl Directory Reference, [15](#)
- rtl/alu_decoder.v, [49](#)
- rtl/control_unit.v, [53](#)
- rtl/control_unit_wrapper.sv, [57](#)
- rtl/op_decoder.v, [57](#)
- run_phase
 - driver, [25](#)
 - monitor, [29](#)
 - scoreboard, [38](#)
 - test, [42](#)
- run_test
 - tb_top.sv, [62](#)
- s_interface, [36](#)
 - Interfaces, [12](#)
- S_TYPE
 - op_decoder.v, [61](#)
- S_type
 - op_decoder.v, [61](#)
- scoreboard, [36](#)
 - ap_imp, [38](#)
 - build_phase, [37](#)
 - check_phase, [37](#)
 - new, [38](#)
 - num_errors, [38](#)
 - run_phase, [38](#)
 - uvm_component_utils, [38](#)
 - write, [38](#)
- scoreboard.sv
 - control_unit_golden, [73](#)
- sequencer, [39](#)
 - parent, [40](#)
 - uvm_component_utils, [40](#)
- SLL
 - alu_decoder.v, [52](#)
- SLT
 - alu_decoder.v, [53](#)
- SLTU
 - alu_decoder.v, [53](#)
- SRA
 - alu_decoder.v, [53](#)
- SRL
 - alu_decoder.v, [53](#)
- SUB
 - alu_decoder.v, [53](#)
- tb Directory Reference, [16](#)
- tb/s_interface.sv, [61](#)
- tb/tb_top.sv, [62](#)
- tb_top.sv
 - forever, [62](#)
 - rst, [62](#)
 - run_test, [62](#)
 - U_CONTROL_UNIT_WRAPPER, [62](#)
 - vif, [62](#)
- test, [40](#)
 - build_phase, [41](#)
 - end_of_elaboration_phase, [41](#)
 - m_sequence, [42](#)
 - new, [42](#)
 - run_phase, [42](#)
 - uvm_component_utils, [42](#)
- Testbench components, [11](#)
- U_ALU_DECODER
 - control_unit.v, [54](#)
- U_CONTROL_UNIT_WRAPPER
 - tb_top.sv, [62](#)
- U_OP_DECODER
 - control_unit.v, [54](#)
- U_type
 - op_decoder.v, [61](#)
- U_TYPE_AUIPC
 - op_decoder.v, [61](#)
- U_TYPE_LUI
 - op_decoder.v, [61](#)
- uvm Directory Reference, [16](#)
- UVM verification components, [13](#)
- uvm/agent.sv, [63](#)
- uvm/control_unit_pkg.sv, [64](#)
- uvm/control_unit_sequence.sv, [64](#), [65](#)
- uvm/coverage.sv, [66](#)
- uvm/driver.sv, [67](#), [68](#)
- uvm/env.sv, [68](#), [69](#)
- uvm/monitor.sv, [70](#)
- uvm/pkt.sv, [71](#), [72](#)
- uvm/scoreboard.sv, [72](#), [74](#)
- uvm/sequencer.sv, [76](#), [77](#)
- uvm/test.sv, [77](#), [78](#)
- uvm_component_utils
 - agent, [18](#)
 - coverage, [22](#)
 - driver, [25](#)
 - env, [27](#)
 - monitor, [29](#)
 - scoreboard, [38](#)
 - sequencer, [40](#)
 - test, [42](#)

- uvm_field_int
 - pkt, [32](#), [33](#)
- uvm_object_utils
 - control_unit_sequence, [20](#)
- uvm_object_utils_begin
 - pkt, [33](#)
- vif
 - tb_top.sv, [62](#)
- write
 - coverage, [23](#)
 - scoreboard, [38](#)
- XOR
 - alu_decoder.v, [53](#)